

上海市智慧交通与文旅信息查询及管理系统

数据库系统原理课程设计报告



学 号 _____

姓 名 _____

专 业 数据科学与大数据技术

授课老师 李文根

目 录

摘 要	II
一、概述.....	1
1.1 课题背景.....	1
1.2 编写目的.....	1
二、需求分析.....	2
2.1 功能需求.....	2
2.2 数据字典.....	2
2.3 数据流图.....	5
三、可行性分析.....	7
3.1 技术可行性.....	7
3.2 应用可行性.....	7
四、概念设计.....	8
4.1 实体.....	8
4.2 实体属性局部 E-R 图	12
4.3 全局 E-R 图	14
五、逻辑设计.....	14
5.1 E-R 图向关系模型的转变	14
5.2 数据模型的优化及规范化设计	17
六、项目管理.....	18
6.1 框架选择.....	18
6.2 开发平台.....	19
七、系统实现.....	20
7.1 系统架构搭建.....	20
7.2 系统逻辑设计	21
7.3 具体功能编写.....	21
7.4 功能测试.....	29
八、总结.....	34
参考文献.....	34
附录	35

摘要

随着文旅产业数字化转型的深入，如何高效管理海量的文旅资源、涉旅企业及动态运行数据，已成为城市精细化治理的重要课题。针对当前文旅数据来源分散、关联性弱及应急响应滞后等痛点，本课程设计以数据库系统原理为指导，设计并实现了一个面向上海市的智慧文旅信息管理系统。

在数据库设计层面，本文详细阐述了从需求分析、概念结构设计到逻辑结构设计的完整过程。采用实体—联系（E-R）模型对景区、酒店、企业、气象及客流等多源异构数据进行统一建模，并通过引入中间关联表巧妙解耦了企业与资源、预警与受灾点之间的多对多关系。设计过程中严格遵循第三范式（3NF）对关系模式进行规范化处理，配合完整性约束与索引优化，构建了结构清晰、低冗余且具备良好扩展性的底层数据架构。

在系统实现层面，项目基于 MySQL 数据库，采用 Python Django 后端框架与 Vue.js 前端框架的前后端分离技术栈，结合 Docker 容器化部署，构建了集数据查询、综合管理与辅助决策于一体的应用平台。系统核心功能涵盖文旅资源的全景数字化建档、基于 GIS 地图的资源分布可视化、客流与入住率的动态监测以及“气象—景点”联动的风险预警机制。

测试结果表明，该数据库系统设计逻辑严密，能够有效支撑复杂业务查询与高并发数据处理，解决了文旅信息管理中的“数据孤岛”问题，为实现智慧文旅的科学管理、风险防控与决策分析提供了坚实的数据基础与技术支持。

关键词：数据库设计；智慧文旅；信息管理系统；E-R 模型；Django

一、概述

1.1 课题背景

随着城市信息化和数字经济的快速发展,文旅产业正从传统的经验驱动模式向以数据驱动和智能决策为核心的方向转型。在超大城市背景下,旅游资源分布广泛、类型多样,涉及景区、酒店、涉旅企业、交通与气象等多源异构数据,传统分散式的信息管理方式已难以满足精细化管理和协同治理的需求。

当前,文旅管理部门在实际工作中普遍面临数据来源分散、更新不及时、关联关系不清晰等问题,尤其在高峰客流调度、恶劣天气预警及应急响应等场景下,缺乏统一的数据支撑平台,难以实现快速、准确的辅助决策。因此,构建一个结构清晰、关联紧密、支持动态数据更新的文旅信息数据库系统,具有重要的现实意义和应用价值。

在此背景下,本课程设计以数据库系统原理为理论基础,结合上海市文旅与气象等开放数据资源,设计并实现一个集信息查询、综合管理与智能关联分析于一体的文旅信息管理系统,通过合理的数据库建模和系统架构设计,为智慧文旅管理提供数据支撑。

1.2 编写目的

本数据库的设计与实现旨在构建一个面向上海市文旅管理与服务场景的综合信息管理平台¹,通过对文旅资源、涉旅企业及气象等多源数据进行统一建模和集中管理,实现数据的规范存储、高效查询与智能关联分析。其具体目的如下:

①实现文旅信息的统一存储与规范管理

通过关系型数据库对景区、酒店、旅游企业等基础信息进行结构化建模,避免数据冗余和信息孤岛问题,提升文旅数据管理的规范性与一致性。

②刻画复杂业务对象之间的关联关系

利用实体—关系模型,准确描述旅游企业与文旅资源之间的运营关系,以及气象预警与受影响景点之间的多对多关系,为业务分析和决策提供可靠的数据支撑。

③支持动态数据的持续更新与查询分析

数据库不仅存储静态的文旅资源信息,还能够管理客流量、酒店入住率及气象变化等动态数据,满足系统对实时监测和历史数据分析的需求。

④提升智慧文旅管理与应急响应能力

通过数据库中多源数据的整合与关联,为文旅管理部门在客流调控、风险预警和应急处置等场景下提供数据依据,增强文旅管理的科学性与前瞻性。

⑤服务于后续系统扩展与应用开发

通过合理的表结构设计和良好的扩展性,为后续基于 Web 的信息查询系统、可视化平台及智能分析功能的实现奠定坚实的数据基础。

此文档通过对实际应用场景的需求分析,说明从概念结构设计(E-R图)到逻辑结构设计,再到物理实现的完整过程,体现数据库系统原理在实际项目中的应用。同时对系统所实

¹ 数据来源为上海市公共数据开放平台(<https://data.sh.gov.cn/>)和中国天气网(<http://www.weather.com.cn>),经纬度信息来自高德开放平台(<https://console.amap.com/>)。

现的核心功能模块进行说明，包括文旅资源管理、企业运营关联、动态数据监测以及气象预警等内容，阐明各功能在数据库层面的实现方式。

二、需求分析

2.1 功能需求

该系统的功能需求主要集中在资源数字化管理、动态运行监测、精细化气象预警以及企业运营关联四个核心维度，具体描述如下：

①文旅资源全景数字化管理

系统首要构建全市文旅资源的“数字底座”。通过后台批量导入与清洗，对景区、酒店等场所进行统一的数字化建档，精细化维护景区等级、最大承载量、酒店星级及房间数等关键静态属性。同时，利用高德地图 API 进行地理编码，将文本地址转化为空间坐标，在前端实现基于 GIS 地图的资源分布可视化查询与详情展示。

②动态运行监测与辅助决策

针对文旅行业的时效性需求，系统需具备实时感知能力。功能涵盖对核心景区实时客流量的持续追踪，以及对酒店经营入住率的动态记录。通过仪表盘形式展示这些“体征”数据，旨在辅助管理者进行人流疏导、“削峰填谷”以及经营策略的科学调整，解决管理滞后的痛点。

③精细化气象服务与应急风控

系统需打破传统的大区域天气预报模式，提供针对具体文旅场所的“一地一策”专属气象服务，展示实时气温与趋势。更关键的是建立闭环的安全响应机制：当气象部门发布预警时，系统应能自动分析预警覆盖范围，智能筛选出受影响的具体景点或酒店并发出示警，实现从被动接收到主动防御的转变。

④旅游企业关联与系统基础服务

为了理清市场主体与资源的关系，系统需构建“企业-资源”实体关系网，明确旅游企业对具体景点或酒店的运营、管理及归属关系。此外，系统还需包含完善的用户中心功能，支持用户注册、登录、画像管理及鉴权，并提供文件上传与通用配置等公共服务接口，保障系统的安全与易用性。

2.2 数据字典

为保证数据库设计的规范性与可维护性，对系统中主要数据对象及其含义进行说明，形成如下数据字典。

2.2.1 文旅资源主表

表 1 POI_Master: 文旅资源主表

字段名	数据类型	主键	说明
poi_id	INT	是	文旅资源唯一编号
poi_name	VARCHAR(255)	否	文旅资源名称

续表 1

字段名	数据类型	主键	说明
poi_type	VARCHAR(100)	否	资源类型（景区、酒店等）
address	VARCHAR(500)	否	地址
longitude	DECIMAL(10,6)	否	经度
latitude	DECIMAL(10,6)	否	纬度
description	TEXT	否	资源描述信息

2.2.2 景区详情表

表 2 ScenicSpot_Details —— 景区详情表

字段名	数据类型	主键	说明
s_id	INT	是	景区详情表主键
scenic_spot_id	INT	否（外键）	对应 poimaster.poi_id
level	VARCHAR(10)	否	景区等级
max_capacity	INT	否	最大承载量

2.2.3 酒店详情表

表 3 Hotel_Details: 酒店详情表

字段名	数据类型	主键	说明
h_id	INT	是	酒店详情表主键
hotel_id	INT	否（外键）	对应 poimaster.poi_id
scenic_spot_detail_id	INT	否（外键）	关联景区详情（如适用）
star_rating	VARCHAR(50)	否	酒店星级
room_count	INT	否	房间数量

2.2.4 涉旅企业信息表

表 4 Tourism_Enterprise: 涉旅企业信息表

字段名	数据类型	主键	说明
enterprise_id	INT	是	企业编号
enterprise_name	VARCHAR(255)	否	企业名称
enterprise_type	VARCHAR(255)	否	企业类型

2.2.5 企业-资源关联表

表 5 Enterprise_POI_Link：企业-资源关联表

字段名	数据类型	主键	说明
e_id	INT	是	关联表主键
enterprise_id	INT	否（外键）	对应 enterpriseinfo.enterprise_id
poi_id	INT	否（外键）	对应 poimaster.poi_id
relationship_type	VARCHAR(255)	否	关系类型（经营/管理等）

2.2.6 动态运行数据表

表 6 Dynamic_Data：动态运行数据表

字段名	数据类型	主键	说明
record_id	INT	是	动态数据编号
poi_id	INT	否（外键）	文旅资源编号
record_time	DATETIME	否	记录时间
passenger_flow	INT	否	实时客流量
occupancy_rate	DECIMAL(5,2)	否	入住率

2.2.7 实时气象表

表 7 Location_Weather：实时气象表

字段名	数据类型	主键	说明
location_id	INT	是	气象记录编号
location_name	VARCHAR(255)	否	地点名称
longitude	DECIMAL(10,6)	否	经度
latitude	DECIMAL(10,6)	否	纬度
poi_id	INT	否（外键）	关联文旅资源
temperature	DECIMAL(5,2)	否	当前温度
conditions	VARCHAR(100)	否	天气状况
update_time	DATETIME	否	更新时间

2.2.8 天气预报表

表 8 Weather_Forecast：天气预报表

字段名	数据类型	主键	说明
forecast_id	INT	是	预报编号

字段名	数据类型	主键	说明
location_id	INT	否（外键）	对应 locationweather.location_id
forecast_date	DATE	否	预报日期
temp_max	DECIMAL(5,2)	否	最高温度
temp_min	DECIMAL(5,2)	否	最低温度
condition_day	VARCHAR(100)	否	白天天气情况

2.2.9 气象预警表

表 9 Weather_Alert: 气象预警表

字段名	数据类型	主键	说明
alert_id	INT	是	预警编号
title	VARCHAR(255)	否	预警标题
level	VARCHAR(50)	否	预警等级
status	VARCHAR(50)	否	预警状态
publish_time	DATETIME	否	发布时间

2.2.10 预警影响资源表

表 10 Alert_Affected_POI: 预警影响资源表

字段名	数据类型	主键	说明
a_id	INT	是	影响关系编号
alert_id	INT	否（外键）	对应 weatheralert.alert_id
poi_id	INT	否（外键）	受影响文旅资源编号

2.3 数据流图

本系统采用“多源异构数据汇聚”的处理模式，将静态的文旅档案、动态的业务运营数据与外部气象数据进行分层处理与融合。以下是各核心处理环节的详细说明：

2.3.1 静态资源清洗与入库

输入流来自原始文件源（CSV）的“静态资源数据”。系统首先对原始数据进行 ETL（抽取-转换-加载）清洗，通过高德地图 API 执行“经纬度解析”，将文本地址转化为空间坐标。

输出流包括基础档案信息（包含景点、酒店、企业基础信息）被存入 D1 文旅资源库，对应数据字典中的 POI_Master、ScenicSpot_Details、Hotel_Details 及 Tourism_Enterprise 表。解析出的企业与资源归属关系被提取并存入 D4 关联关系库，对应 Enterprise_POI_Link 表。

2.3.2 动态运行监测处理

输入流来自运营端的实时“客流/入住率数据”。系统读取 D1 文旅资源库 中的位置信息与 ID 进行校验，确保数据归属的准确性，并打上时间戳。

输出流包括清洗后的时序数据被写入 D2 动态运营库，对应 Dynamic_Data 表，用于支撑后续的历史趋势分析与实时展示。

2.3.3 气象服务与预警关联

输入流是通过第三方气象 API 实时拉取的“实时/预报/预警”数据。将常规气象数据格式化存储。当接收到预警信息时，系统基于 GIS 算法计算预警范围，自动匹配位于该范围内的 POI。

输出流包括实时气象、预报及预警元数据存入 D3 气象信息库，对应 Location_Weather、Weather_Forecast 和 Weather_Alert 表。计算得出的受灾影响关系被写入 D4 关联关系库，对应 Alert_Affected_POI 表，建立“预警-资源”的动态映射。

2.3.4 综合查询与可视化服务

输入流来自用户或管理员的“查询请求”。作为数据应用层，该模块执行跨库联合查询：

- 从 D1 获取资源详情与空间位置；
- 从 D2 获取实时的 passenger_flow（客流）与 occupancy_rate（入住率）；
- 从 D3 获取即时气象状态；
- 从 D4 获取企业归属及当前生效的预警影响。

数据融合后，以 GIS 地图、统计报表或弹窗警告的形式反馈给前端界面。

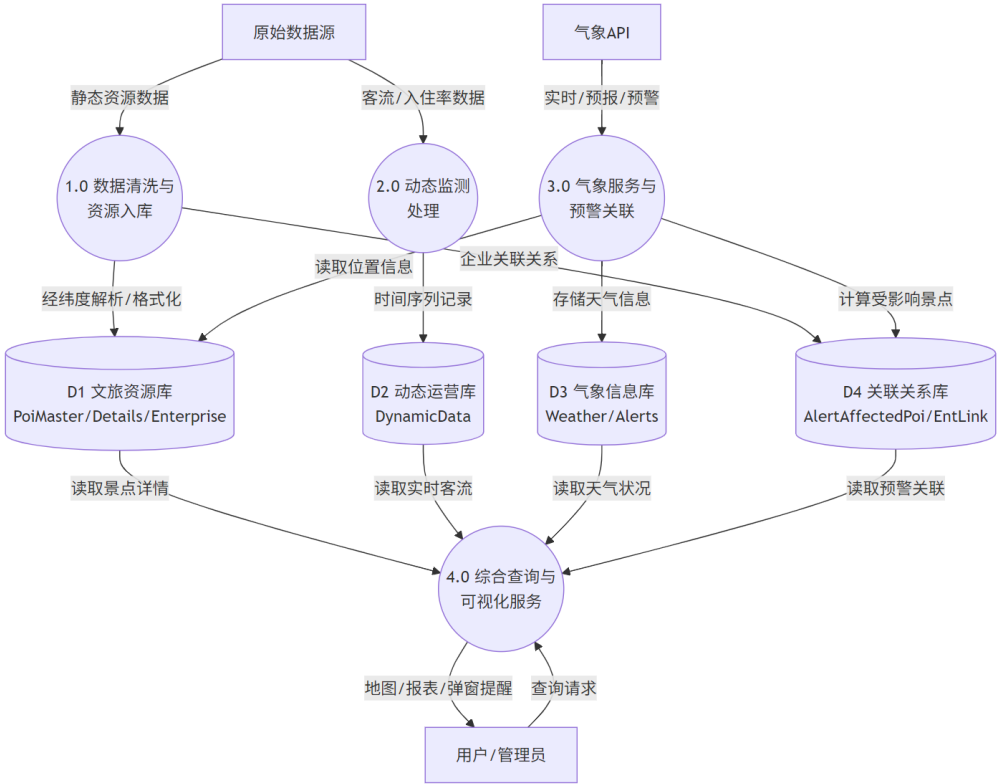


图 1 数据流图

三、可行性分析

3.1 技术可行性

从技术角度来看，本数据库系统的设计与实现具有较高的可行性。

首先，在数据库模型设计方面，本系统基于关系数据库理论，采用 E-R 模型对文旅资源、涉旅企业、气象信息及其关联关系进行建模。通过主键与外键机制，合理描述了一对一、一对多以及多对多关系，结构清晰、约束明确，符合数据库系统原理课程中关于规范化设计和完整性约束的要求。

其次，在数据库管理系统选择方面，本系统采用 MySQL 作为后端数据库。MySQL 具备成熟稳定、性能可靠、对多表关联查询支持良好等优势，能够满足文旅资源信息存储、动态数据记录以及复杂查询的需求。同时，MySQL 对事务处理和索引机制的支持，为系统数据一致性和查询效率提供了保障。

再次，在系统实现层面，本系统基于 Django 框架进行开发，利用其内置的 ORM 技术实现数据库表与程序模型之间的映射，降低了直接编写 SQL 的复杂度，并有效减少了数据库操作错误的风险。Django 自动生成的用户、权限及系统管理相关表，为数据库系统提供了稳定的运行支撑，使业务数据管理与系统管理相互独立、互不干扰。

最后，从数据来源与处理角度看，系统所需的文旅基础数据、气象数据及地理信息均可通过已有的开放数据平台或第三方 API 获取，并能够以结构化形式存入数据库，数据获取方式明确、处理流程清晰，技术实现路径成熟可行。

综上所述，无论从数据库设计方法、数据库管理系统选择，还是系统实现技术角度，本系统在技术上均具有较高的可行性。

3.2 应用可行性

从应用角度来看，本数据库系统能够较好地契合智慧文旅管理与信息服务的实际需求，具有明确的应用价值。

首先，本系统面向文旅管理与服务场景，能够对景区、酒店及涉旅企业等信息进行集中管理，解决了传统管理模式信息分散、更新不及时的问题，为管理部门提供统一的数据支撑平台，具备良好的实用性。

其次，通过对动态客流数据和气象信息的持续记录与关联分析，数据库能够为客流调控、风险预警及应急管理提供数据基础，提升文旅管理的科学性和前瞻性。这种以数据为核心的管理方式，符合当前智慧城市与智慧文旅的发展趋势。

再次，系统数据库结构具有较好的扩展性。在不破坏现有表结构的前提下，可通过增加业务表或字段，支持更多类型的文旅资源或分析需求，满足系统后续功能拓展与规模扩大的应用场景。

最后，从用户使用角度看，数据库为上层应用系统提供了稳定的数据接口支持，可服务于管理端、业务端及公众查询等多种应用形式，应用对象明确，使用场景清晰，具备良好的推广与应用前景。

四. 概念设计

4.1 实体

根据系统需求分析，围绕上海市智慧文旅信息查询与管理的业务场景，对系统中涉及的核心数据对象进行抽象，形成如下主要实体。各实体在概念层面仅描述其业务含义及存在必要性，具体属性与实现细节在后续逻辑设计中说明。

4.1.1 文旅资源实体（POI）

文旅资源实体用于描述系统中统一管理的各类文旅对象，是系统的核心实体。该实体涵盖景区、酒店等不同类型的文旅资源，负责承载资源的基础信息与空间位置属性，为动态数据监测、企业关联及气象服务提供统一的数据载体。

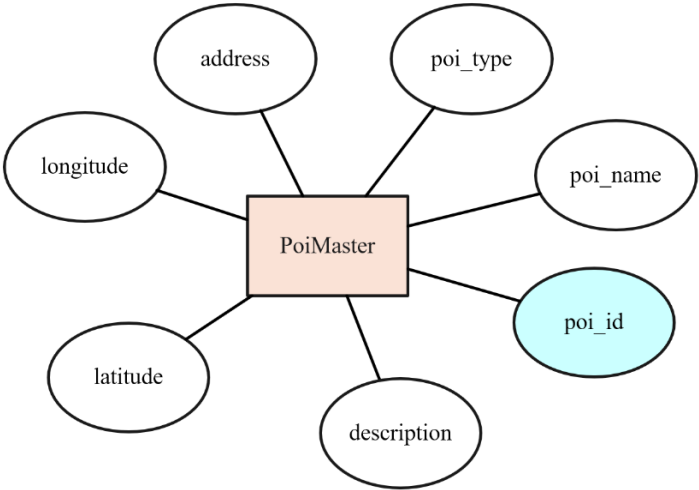


图 2 文旅资源实体介绍图

4.1.2 景区实体

景区实体是对文旅资源实体的细化扩展，用于描述景区所特有的业务属性，如景区等级和最大承载量等。该实体与文旅资源实体之间存在一对一的对应关系，用于区分不同类型文旅资源的差异化特征。

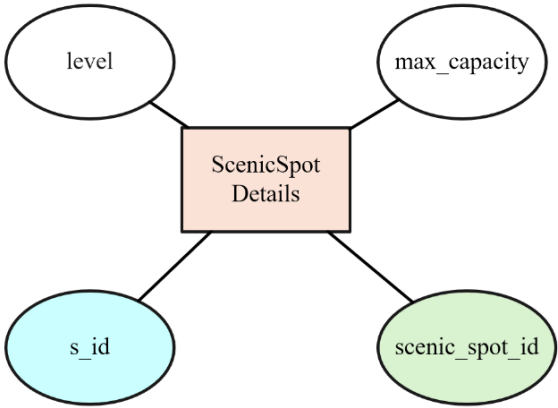


图 3 景区实体介绍图

4.1.3 酒店实体

酒店实体同样是文旅资源实体的扩展实体，用于描述酒店的特有属性，如星级标准、房间数量等。通过与文旅资源实体的关联，实现对酒店信息的规范化管理。

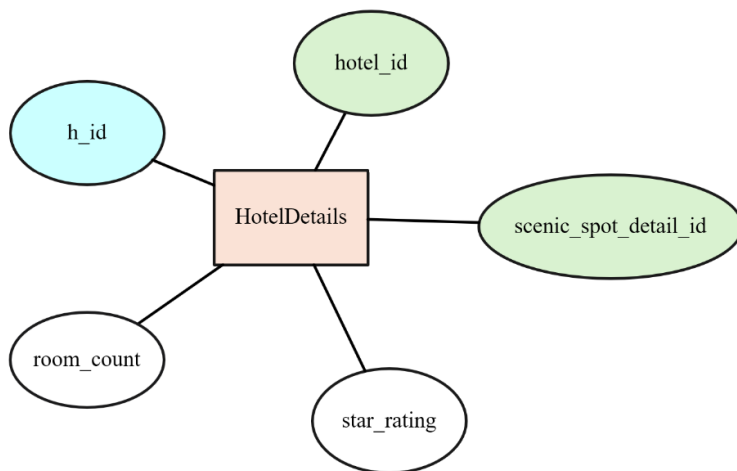


图 4 酒店实体介绍图

4.1.4 涉旅企业实体

涉旅企业实体用于描述参与文旅资源运营或管理的企业主体。该实体记录企业的基本信息，是建立企业与文旅资源运营关系的重要基础。。

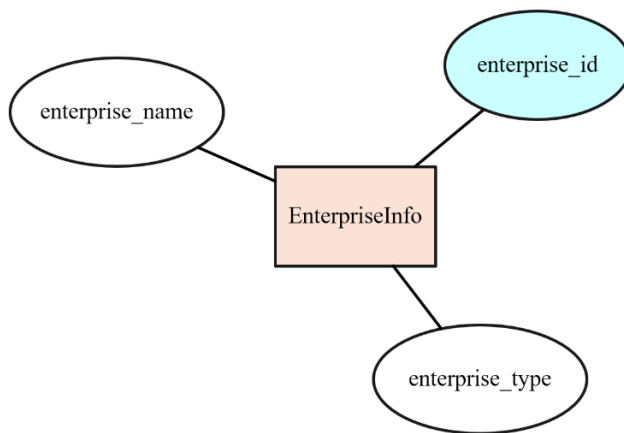


图 5 涉旅企业实体介绍图

4.1.5 企业-资源关联实体

企业—资源关联实体用于描述涉旅企业与文旅资源之间的业务关系，如经营、管理或其他合作关系。该实体主要用于解决企业与文旅资源之间的多对多关系，使系统能够准确表达复杂的运营归属结构。

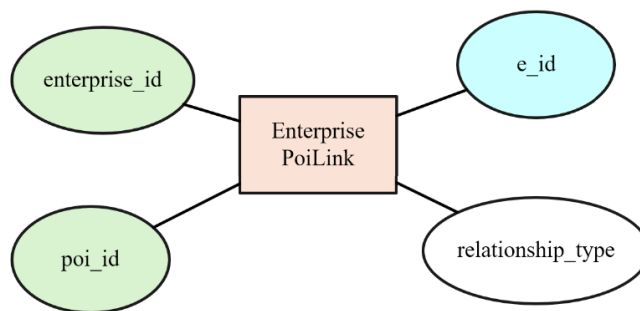


图 6 企业-资源关联实体介绍图

4.1.6 动态运行数据实体

动态运行数据实体用于描述文旅资源在运行过程中产生的时间序列数据，如景区客流量、酒店入住率等。该实体以文旅资源为关联对象，支持对文旅运行状态的实时监测与历史分析。

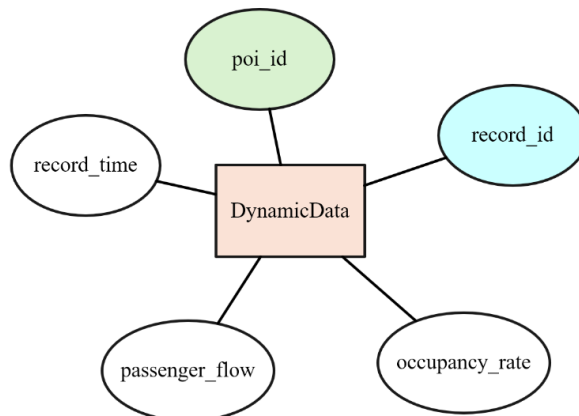


图 7 动态运行数据实体介绍图

4.1.7 气象位置实体

气象位置实体用于描述与文旅资源相关联的气象观测点，负责承载地理位置与实时气象数据，是气象服务与文旅资源关联的桥梁实体。

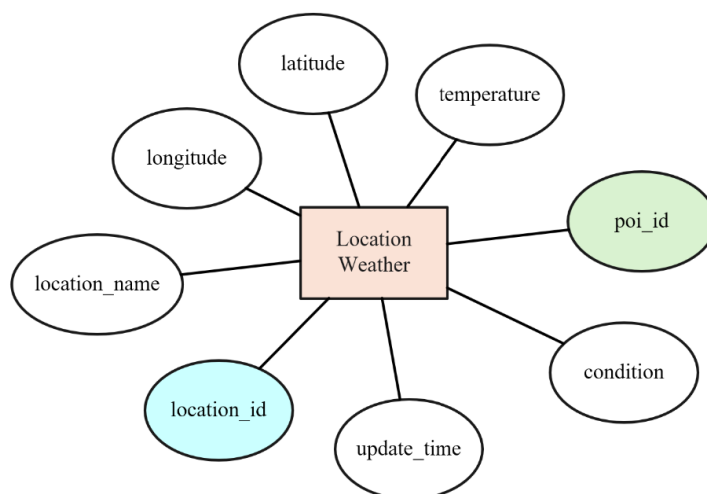


图 8 气象位置实体介绍图

4.1.8 天气预报实体

天气预报实体用于描述未来时间段内的天气预测信息，与气象位置实体相关联，为文旅活动规划和风险评估提供数据支持。

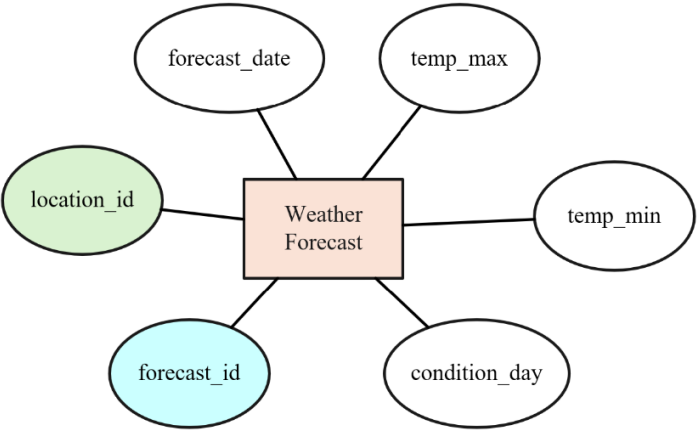


图 9 天气预报实体介绍图

4.1.9 气象预警实体

气象预警实体用于描述气象部门发布的各类预警信息，包括预警等级、状态及发布时间等内容，是系统实现风险提示与应急响应的重要数据基础。

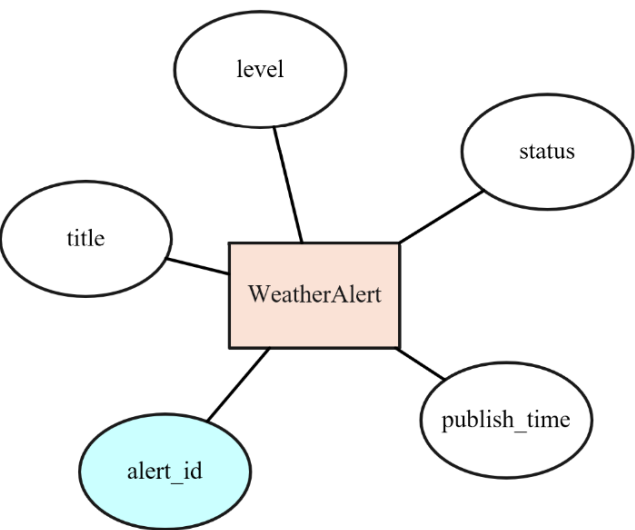


图 10 气象预警实体介绍图

4.1.10 预警影响资源实体

预警影响资源实体用于描述某次气象预警所影响的具体文旅资源，用于建立气象预警与文旅资源之间的对应关系，实现对受影响区域和场所的精准识别。

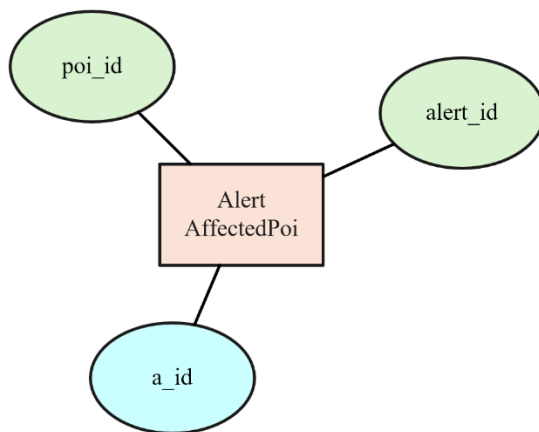


图 11 预警影响资源实体介绍图

4.2 实体属性局部 E-R 图

4.2.1 文旅资源基础信息

这是系统的核心底座，展示了“通用资源”与“专用资源”的继承/扩展关系。POI_Master 存储所有场所的公共属性，而 ScenicSpot_Details 和 Hotel_Details 分别存储景区和酒店的特有属性。

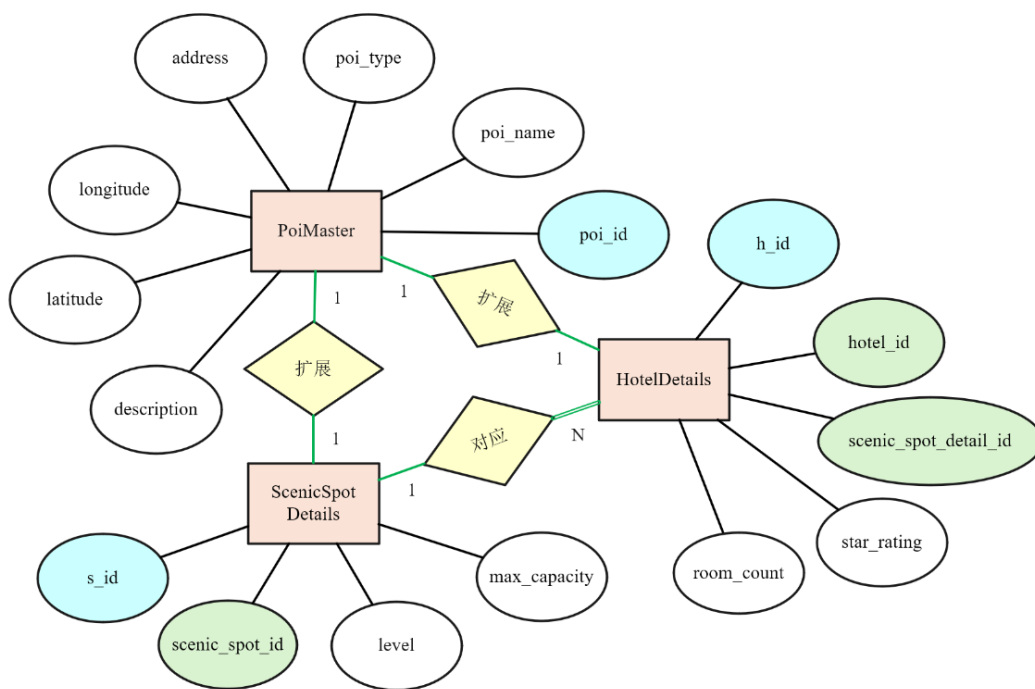


图 12 文旅资源基础信息局部 E-R 图

4.2.2 企业运营关联

展示市场主体(企业)与文旅资源之间的多对多关系。通过中间表 Enterprise_POI_Link, 解决了一个企业管理多个资源, 或一个资源被多个企业(如物业、运营方)关联的复杂业务。

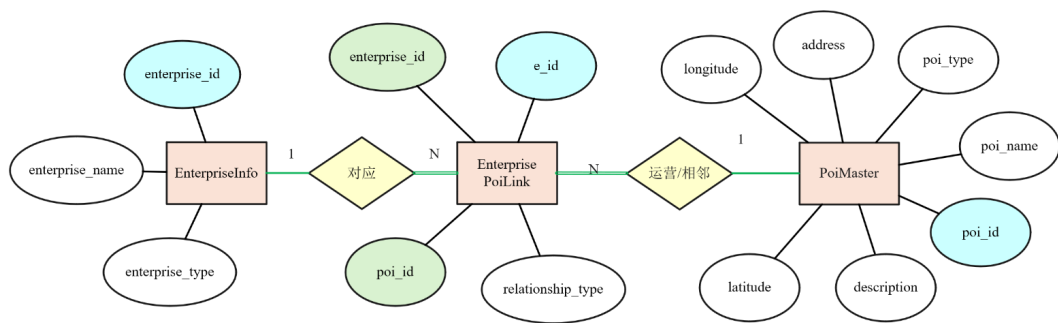


图 13 企业运营关联局部 E-R 图

4.2.3 动态运行监测

展示资源与时间序列数据的关系。Dynamic_Data 是系统的“心跳”，记录了 POI_Master 在不同时间点的客流与经营状况。

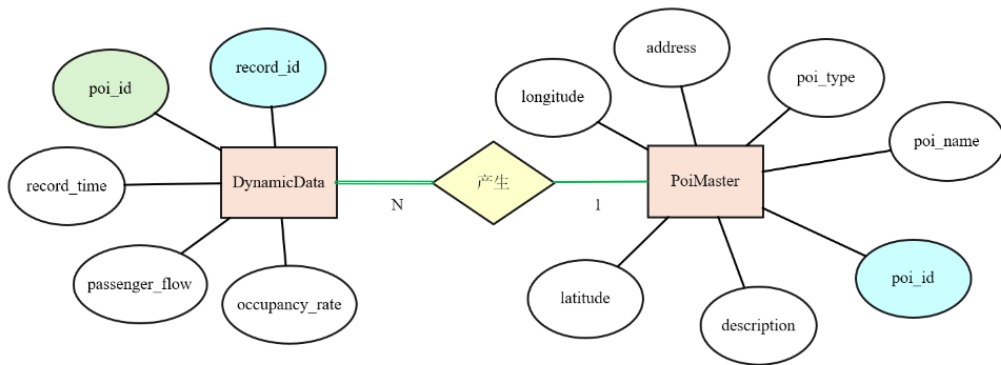


图 14 动态运行监测局部 E-R 图

4.2.4 气象服务与预警

该模型构建了全链路气象风控体系：左侧通过实时气象与预报数据的 1:N 关联，覆盖全时间维度监测；右侧利用 Alert_Affected_POI 中间实体，将预警信息精确映射至具体资源，解决“灾害-景点”的多对多业务难题，实现“一地一策”的精准风险示警。

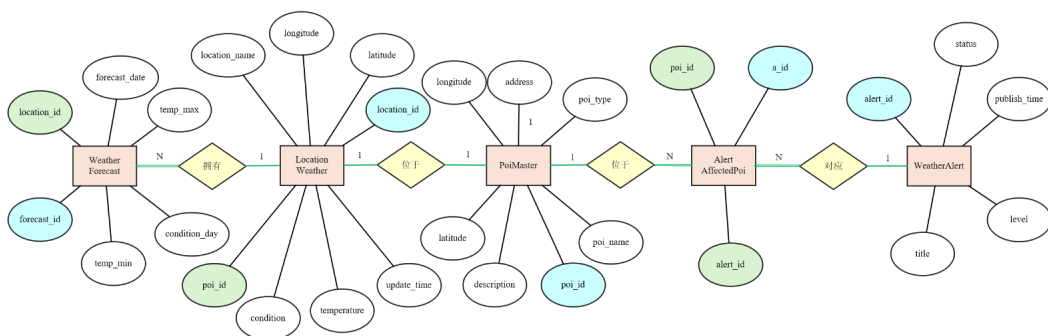


图 15 气象服务与预警局部 E-R 图

4.3 全局 E-R 图

本系统 E-R 模型设计以 PoiMaster 为核心实体构建全景资源底座，采用“基础+扩展”的垂直分表策略，通过 ScenicSpotDetails 和 HotelDetails 扩展实体实现对不同类型文旅场所差异化属性的精细管理。为解决复杂的业务关联，模型引入了 EnterprisePoiLink 和 AlertAffectedPoi 两个中间关联实体，巧妙解耦了“涉旅企业与资源运营”及“气象预警与受灾景点”这两个多对多（M:N）关系。同时，通过 DynamicData 实体记录客流与入住率的时序数据，配合 LocationWeather 与 WeatherForecast 构成的气象监测体系，共同形成了集静态档案、动态监测、企业图谱与风险管控于一体的闭环数据架构。

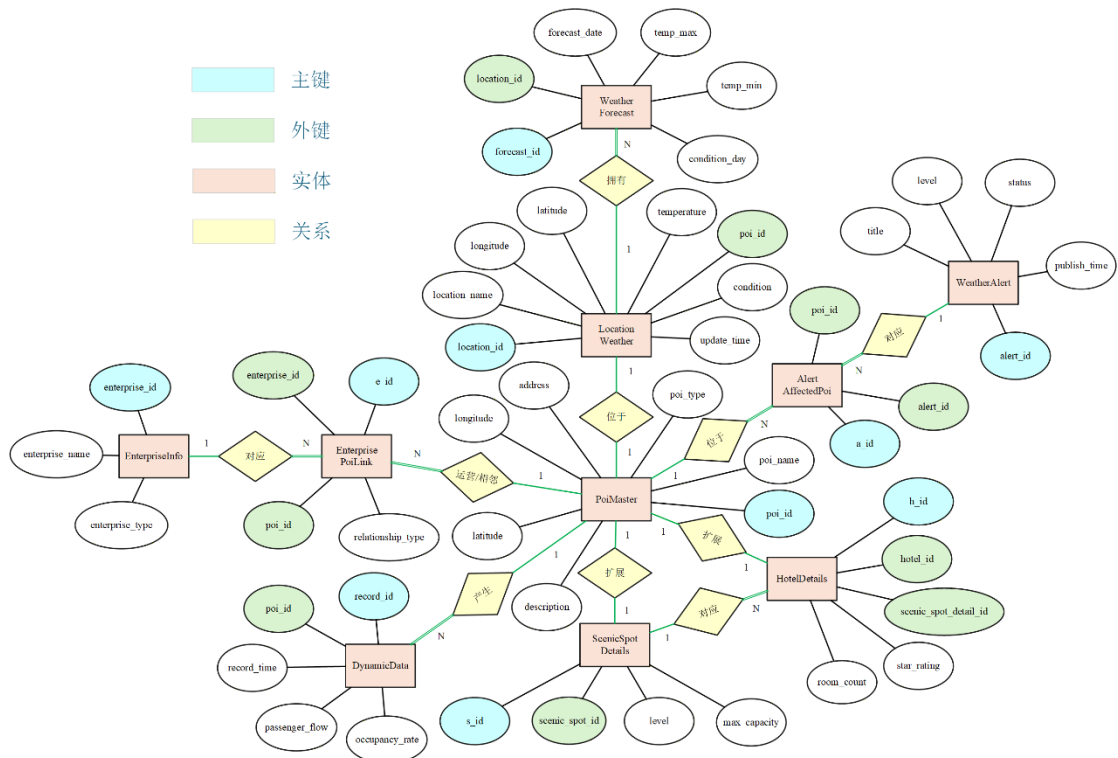


图 16 全局 E-R 图

五、逻辑设计

5.1 E-R 图向关系模型的转变

在完成概念结构设计并构建系统 E-R 图的基础上，需要将 E-R 模型转化为关系模型，以便在关系数据库管理系统中进行实现。转换过程中遵循关系数据库设计的基本原则，将实体、属性及实体间联系映射为关系模式，并通过主键和外键来体现实体之间的约束关系。

5.1.1 实体向关系模式的转换

表 11 实体和关系模式对应表

实体	关系模式
文旅资源实体（POI）	POIMaster (<i>poi_id</i> , poi_name, poi_type, address, longitude, latitude, description)

实体	关系模式
景区实体	ScenicSpotDetails (<i>s_id</i> , level, max_capacity, scenic_spot_id)
酒店实体	HotelDetails (<i>h_id</i> , star_rating, room_count, hotel_id, scenic_spot_detail_id)
涉旅企业实体	EnterpriseInfo (<i>enterprise_id</i> , enterprise_name, enterprise_type)
动态运行数据实体	DynamicData (<i>record_id</i> , record_time, passenger_flow, occupancy_rate, poi_id)
气象位置实体	LocationWeather (<i>location_id</i> , location_name, longitude, latitude, temperature, conditions, update_time, poi_id)
天气预报实体	WeatherForecast (<i>forecast_id</i> , forecast_date, temp_max, temp_min, condition_day, location_id)
气象预警实体	WeatherAlert (<i>alert_id</i> , title, level, status, publish_time)

5.1.2 联系向关系模式的转换

在将 E-R 图转换为关系模型的过程中, 需要根据实体之间联系的基数类型, 采用不同的转换策略。系统中主要涉及一对一 (1:1)、一对多 (1:N) 以及多对多 (N:M) 三种联系类型, 具体说明如下。

① 一对一 (1:1) 联系的转换

一对一联系表示一个实体的一个实例至多只对应另一个实体的一个实例, 反之亦然。在本系统中, 该类联系主要用于描述文旅资源实体与其扩展实体之间的对应关系。

例如:

- 文旅资源 (POI) ↔ 景区详情 (ScenicSpotDetails)
- 文旅资源 (POI) ↔ 酒店详情 (HotelDetails)

对于 1:1 联系, 通常有以下两种转换方式: 将其中一个实体的主键作为外键加入到另一个实体中或将两者合并为一个关系模式。

本系统中, 为保证结构清晰并避免无关属性混杂, 采用独立扩展表的方式, 将文旅资源的主键 poi_id 作为外键引入到景区详情表和酒店详情表中。

② 一对多 (1:N) 联系的转换

一对多联系表示一个实体的一个实例可以对应另一个实体的多个实例, 但反方向仅能对应一个实例。这类联系在业务系统中最为常见, 尤其适用于时间序列和记录型数据。

例如:

- 文旅资源 (POI) ↔ 动态运行数据 (DynamicData)
- 文旅资源 (POI) ↔ 气象位置 (LocationWeather)
- 气象位置 (LocationWeather) ↔ 天气预报 (WeatherForecast)

对于 1:N 联系，将“一”端实体的主键作为外键加入到“多”端实体中，从而在关系模型中体现实体间的从属关系。

③多对多（N:M）联系的转换

（1）联系特征说明

多对多联系表示一个实体的一个实例可以对应另一个实体的多个实例，反之亦然。这类联系无法通过简单的外键直接表示，需要引入独立的关联实体。

例如：

- 涉旅企业（Enterprise）↔文旅资源（POI）
- 气象预警（WeatherAlert）↔文旅资源（POI）

一个企业可以运营多个文旅资源，而一个文旅资源也可能由多个企业参与运营。对于 N:M 联系，引入中间关系模式，该关系模式至少包含：两端实体的主键作为外键；联系自身所具有的属性（如关系类型）。

关系模型表示为：

EnterprisePOILink (*e_id*, relationship_type, enterprise_id, poi_id)

AlertAffectedPOI (*a_id*, alert_id, poi_id)

通过中间表的方式，将多对多联系拆分为两个一对多联系，从而满足关系模型的表达要求。

通过对一对一、一对多及多对多联系的分析与转换，系统中各类实体关系均被规范地映射为关系模型。该转换过程既保持了概念模型的语义完整性，又满足了关系数据库对结构化存储和完整性约束的要求，为后续数据库实现奠定了良好的逻辑基础。

5.1.3 主键与外键的确定

在关系模型中，各关系模式均设置唯一的主键，用于标识元组的唯一性；同时，通过外键约束来保证实体之间引用关系的正确性。例如：

表 12 外键约束汇总表

子表	外键字段	被引用表	被引用字段
ScenicSpotDetails	scenic_spot_id	POIMaster	poi_id
HotelDetails	hotel_id	POIMaster	poi_id
EnterprisePOILink	enterprise_id	EnterpriseInfo	enterprise_id
EnterprisePOILink	poi_id	POIMaster	poi_id
DynamicData	poi_id	POIMaster	poi_id
LocationWeather	poi_id	POIMaster	poi_id
WeatherForecast	location_id	LocationWeather	location_id
AlertAffectedPOI	alert_id	WeatherAlert	alert_id
AlertAffectedPOI	poi_id	POIMaster	poi_id

5.2 数据模型的优化及规范化设计

在完成 E-R 图向关系模型的转换后,需要对所得关系模式进行优化与规范化设计,以减少数据冗余,避免更新异常,提高数据库的一致性和可维护性。本系统在逻辑结构设计阶段,依据关系数据库规范化理论,对数据模型进行了系统分析和优化。

5.2.1 第一范式(1NF)分析

第一范式要求关系中的每个属性都是不可再分的原子值,不允许出现重复组或多值属性。

在本系统中,各关系模式的属性均为原子属性,例如文旅资源名称、地址、经纬度、企业类型、客流量等字段均为单一取值,不存在属性值的进一步拆分需求。同时,动态运行数据和天气预报等时间序列信息通过独立记录的方式进行存储,避免了在单个字段中存放多个时间点的数据。因此,系统中各数据表均满足第一范式的要求。

5.2.2 第二范式(2NF)分析

第二范式要求在满足第一范式的基础上,所有非主属性必须完全依赖于主键,不能只依赖于主键的一部分。

在本系统中,各实体表均设置了单一主键(如 `poi_id`、`enterprise_id`、`record_id` 等),不存在复合主键的情况。所有非主属性均依赖于对应实体的主键,例如文旅资源的名称、类型和位置属性完全依赖于 `poi_id`,动态运行数据中的客流量和入住率完全依赖于 `record_id`。

对于多对多关系引入的中间表,其业务属性(如关系类型)依赖于该关联关系本身,而不是某一端实体的部分属性。因此,系统关系模式不存在部分函数依赖问题,满足第二范式。

5.2.3 第三范式(3NF)分析

第三范式要求在满足第二范式的基础上,关系模式中不存在非主属性对主键的传递函数依赖。即任意非主属性都必须直接依赖于主键,而不能通过其他非主属性间接依赖主键。

本系统在设计过程中,针对可能产生传递依赖的场景进行了重点分析,并通过实体拆分和关系分离的方式进行优化,具体说明如下。

① 文旅资源相关表的 3NF 分析

在概念设计阶段,文旅资源同时包含景区与酒店两类对象。如果将景区等级、最大承载量、酒店星级、房间数量等属性统一存储在文旅资源表中,将会导致如下问题:景区属性与酒店属性同时依赖于资源类型;非主属性(如景区等级)并不适用于所有文旅资源;出现“资源类型→特有属性”的传递依赖。

为消除该类传递依赖,系统将文旅资源的通用属性与类型特有属性进行拆分(表 11),拆分后,景区与酒店的特有属性均直接依赖于各自表的主键,不再通过资源类型间接依赖 `poi_id`,从而满足第三范式。

② 涉旅企业与运营关系的 3NF 分析

若将企业信息与其运营的文旅资源信息直接存储在同一关系模式中,将导致企业名称、企业类型等属性随着资源关系变化而重复出现,形成如下传递依赖:

关联记录→企业编号→企业名称、企业类型

为避免该问题,系统将企业实体与企业-资源关系实体分离(表 11),其中,企业名称和企业类型仅依赖于 `enterprise_id`,而不依赖于关联关系本身,从而消除了非主属性之间的传

递依赖。

③动态运行数据的 3NF 分析

动态运行数据表用于存储文旅资源在不同时间点的运行状态。若在该表中直接存储文旅资源名称、类型等信息，将会产生如下依赖链：

记录编号→文旅资源编号→文旅资源名称、类型

因此，本系统在动态运行数据表中仅保留与运行状态直接相关的属性，并通过外键引用文旅资源主键（表 11），这样，所有非主属性均直接依赖于 record_id，不存在通过 poi_id 产生的传递依赖。

④气象数据相关表的 3NF 分析

气象信息涉及实时天气、天气预报和气象预警等不同业务对象。若将这些信息集中存储在单一关系模式中，将导致预报信息、预警信息之间相互冗余，并产生复杂的传递依赖。

因此，系统将气象数据拆分（表 11），拆分后，各表中的非主属性均直接依赖于对应实体的主键，不存在通过其他非主属性产生的传递依赖，满足第三范式要求。

通过上述分析与拆分，本系统在以下方面满足第三范式要求：消除了非主属性对主键的间接依赖；避免了属性因业务类型不同而产生的条件依赖；减少了数据冗余，防止插入、删除和更新异常；提高了数据库结构的清晰性和可维护性。

通过将文旅资源通用属性与类型特有属性分离，并将企业、气象及动态数据等对象独立建模，系统有效消除了非主属性之间的传递依赖，整体关系模式满足第三范式要求。

六、项目管理

6.1 框架选择

本项目在开发过程中，综合考虑系统规模、功能需求以及课程设计的实践要求，选用成熟、稳定且易于维护的开发框架，以保证系统实现的可行性和开发效率。

后端框架选用 Django。这是基于 Python 的高层 Web 开发框架，内置完善的 ORM、用户认证和后台管理机制，能够有效简化数据库操作和系统管理工作。通过 Django ORM，可以将关系模型与程序对象进行映射，减少直接编写 SQL 语句的复杂度，同时降低数据库操作错误的风险，便于数据库结构的维护与调整。此外，Django 自动生成的认证和权限管理模块，有助于实现系统的用户管理和访问控制，为项目的稳定运行提供支撑。

在接口层设计上，项目结合 Django REST Framework (DRF) 构建统一的数据访问接口，使数据库中的文旅资源、动态运行数据和气象信息能够以标准化的方式对外提供服务，增强系统的扩展性和模块解耦能力。

前端框架选用 Vue.js。Vue 具有组件化程度高、学习成本低和开发效率高等特点，适合构建数据展示和交互界面。通过前后端分离的架构设计，前端专注于数据可视化和用户交互，后端专注于业务逻辑和数据库管理，有效提升了系统整体的可维护性。

本项目使用 Node.js 环境，在本项目的实现过程中，为提高系统部署的一致性和环境可复现性，引入 Docker 容器化技术 对后端服务和数据库环境进行统一管理。Docker 通过容器的方式对应用运行环境进行封装，使系统运行所依赖的 Python 版本、第三方库以及数据

库配置保持一致，有效避免了因环境差异导致的运行问题。该方式尤其适用于课程设计项目在不同设备或不同阶段进行部署和演示的场景。



图 17 项目环境和框架选择

6.2 开发平台

本项目在开发与测试阶段主要基于通用的软件与硬件平台完成，平台环境稳定，配置要求适中，具有良好的可复现性。

在开发环境方面，项目基于 Windows 操作系统进行开发，使用 Python 作为主要开发语言，后端数据库采用 MySQL。MySQL 作为成熟的关系型数据库管理系统，能够稳定支持多表关联查询、完整性约束以及事务处理，满足系统对数据一致性和可靠性的要求。

在开发工具方面，前后端代码编写与调试主要使用 Visual Studio Code。数据库管理与调试通过 MySQL Workbench 可视化工具完成，便于对数据库结构和数据进行直观管理。

在运行与测试方面，项目通过本地服务器完成系统部署与功能验证，结合浏览器进行访问测试，能够较为真实地模拟系统的实际运行环境，为系统功能完善和性能优化提供保障。



图 18 项目开发平台

七、系统实现

7.1 系统架构搭建

本系统采用前后端分离的三层体系架构，结合 Docker 容器化部署方式，对系统各功能模块进行合理划分，以提高系统的可维护性、扩展性和运行稳定性。整体架构由前端展示层、后端业务逻辑层和数据库数据层构成。

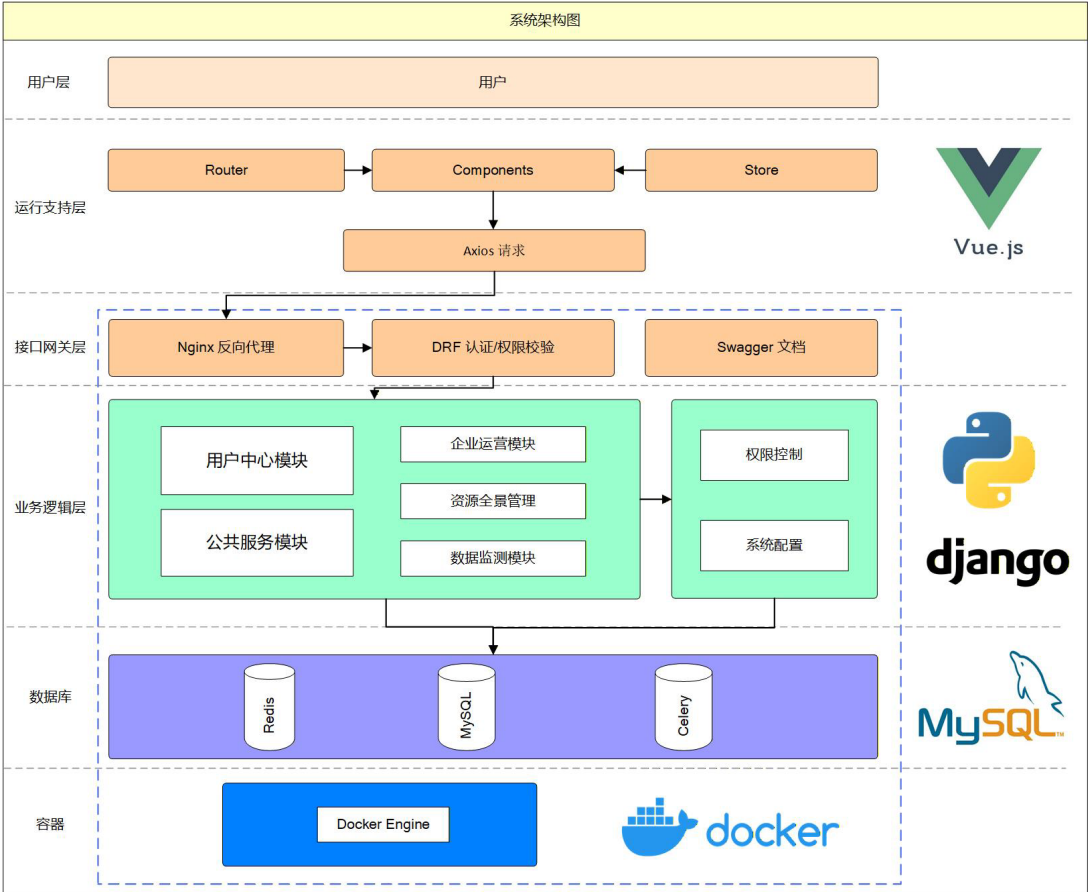


图 19 系统架构图

在前端展示层，系统采用 Vue.js 框架实现用户界面与交互逻辑，主要负责文旅资源信息展示、数据查询结果呈现以及用户操作的响应处理。前端通过组件化方式组织页面结构，并通过 HTTP 请求与后端接口进行数据交互，实现界面展示与业务逻辑的解耦。

在后端业务逻辑层，系统基于 Django 框架构建核心服务，负责处理前端请求、执行业务逻辑并进行数据访问控制。后端通过 Django ORM 实现对 MySQL 数据库中各关系表的操作，将数据库的关系模型映射为程序对象，减少直接操作数据库带来的复杂性和错误风险。同时，后端通过统一的接口设计，对文旅资源、动态运行数据及气象信息等业务功能进行集中管理。

在数据存储层，系统采用 MySQL 关系型数据库 作为数据管理平台，负责存储系统运行过程中产生的各类业务数据。通过主键、外键及完整性约束的设置，保证数据的一致性和完整性，为系统稳定运行提供可靠的数据支撑。

在系统部署与运行管理方面，引入 Docker 容器化技术，将前端服务、后端服务和数据

库服务分别部署在独立容器中，实现系统组件的解耦和统一管理。通过 Docker，可以快速完成系统环境的搭建，保证不同环境下系统运行的一致性，提高系统的可移植性和部署效率。

7.2 系统逻辑设计

系统逻辑设计是在系统架构搭建的基础上，对各功能模块的实现流程和数据处理逻辑进行规划与说明。本系统围绕文旅资源管理、企业关联、动态数据监测以及气象服务等核心业务，构建了清晰、完整的功能实现逻辑。

系统以文旅资源作为核心业务对象。用户通过前端界面发起查询或管理请求，后端接收请求后，首先对请求参数进行合法性校验，然后通过 Django ORM 访问文旅资源主表，获取资源的基础信息。

当资源类型为景区或酒店时，系统根据资源标识，进一步访问对应的扩展表（景区详情表或酒店详情表），完成对资源详细信息的整合与返回。该逻辑设计保证了不同类型文旅资源在统一管理的同时，又能体现其差异化特征。

在企业信息管理模块中，系统分别维护涉旅企业信息和企业-资源关联关系。当需要查询某一企业所运营的文旅资源时，系统通过企业-资源关联表，获取该企业关联的资源标识，再进一步查询文旅资源主表，完成企业与文旅资源之间关系的映射。该逻辑设计有效支持企业与文旅资源之间的多对多关系，避免在单一表中存储冗余信息，提高了系统的数据一致性和可扩展性。

系统针对文旅资源的运行状态，设计了动态数据采集与展示逻辑。每条动态运行数据均与具体的文旅资源相关联，系统在接收到动态数据请求时，根据资源标识查询动态数据表，并按照时间顺序对数据进行筛选和整理。通过该逻辑，系统能够实现对文旅资源运行状态的实时监测和历史数据分析，为后续的数据统计和趋势分析提供支持。

系统在气象服务模块中，首先建立文旅资源与气象位置之间的对应关系。天气预报信息和气象预警信息均以气象位置或文旅资源为关联对象进行存储。

在查询过程中，系统先根据文旅资源获取对应的气象位置信息，再进一步查询天气预报和预警表。当发生气象预警时，系统通过预警影响资源关联表，确定受影响的具体文旅资源，从而实现气象风险的精准提示。

系统采用前后端分离的设计方式。前端通过 HTTP 请求向后端发送功能请求，后端统一处理请求并返回结构化数据结果。前端根据返回结果完成数据展示和用户交互。该逻辑设计减少了前后端之间的耦合程度，使系统在功能扩展和界面调整时更加灵活。

7.3 具体功能编写

7.3.1 数据库实现

本系统数据库采用 MySQL 关系型数据库，数据库结构依据前期完成的概念设计与逻辑设计进行实现，整体以文旅资源（POI）为核心，围绕企业信息、动态运行数据以及气象信息等业务模块进行表结构设计。

我通过 SQL CREATE TABLE 语句定义了所有业务数据表。在业务数据层面，系统主要包括文旅资源主表、景区详情表、酒店详情表、企业信息表、企业-资源关联表、动态运行数据表、气象位置表、天气预报表、气象预警表以及预警影响资源关联表等。各数据表之间

通过主键和外键建立明确的引用关系，保证数据之间的逻辑一致性。

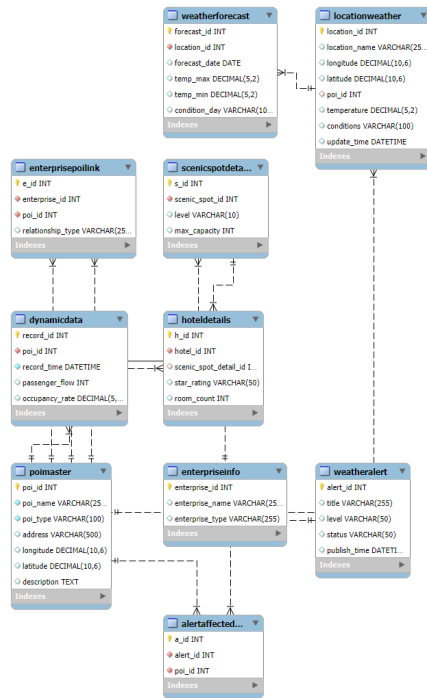


图 19 数据库模型图

此外，系统同时使用了 Django 框架自动生成的用户管理、权限控制及日志相关表，用于实现系统用户认证、权限分配和操作记录管理。这些表与业务表共同构成完整、规范的数据库体系。

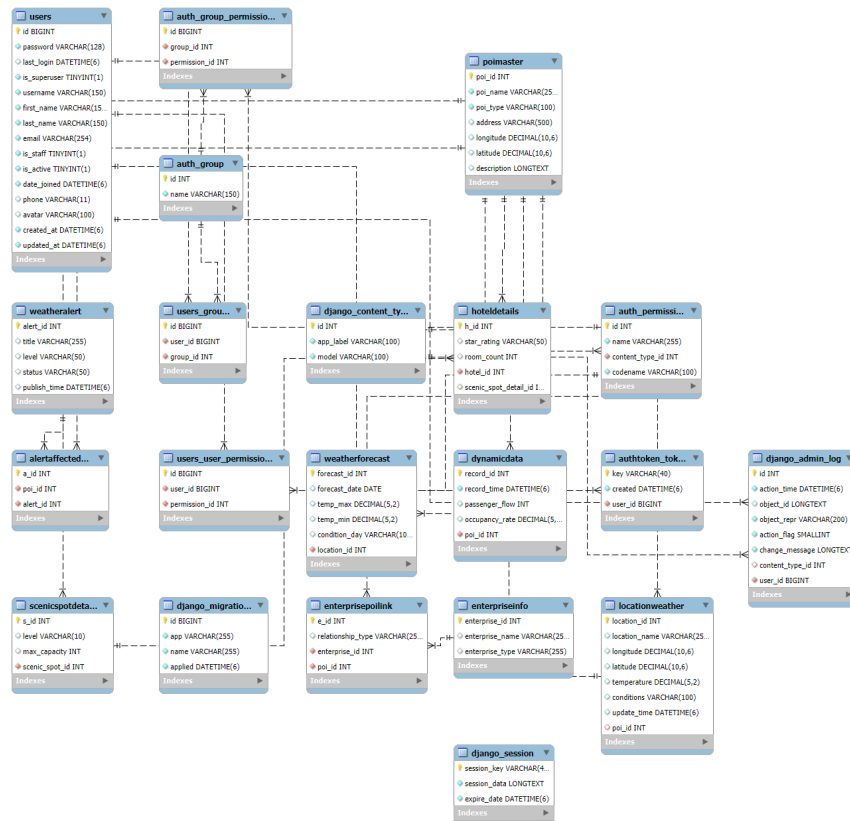


图 20 添加用户管理、权限控制及日志相关表的数据库模型图

7.3.2 后端实现方式

本系统后端基于 Python Django Web 框架构建，采用前后端分离的开发模式。后端核心职责包括：基于 ORM（对象关系映射）构建数据模型、利用 Django Admin 提供高效的后台管理功能，以及通过 Django REST Framework 对外提供标准化的 RESTful API 接口。同时支持 Swagger/ReDoc 自动生成 API 文档，便于前后端协作。

①数据模型与 ORM 实现

在后端开发初期，系统严格依据数据库设计规范，在 Django 框架中定义了与数据库表结构一一对应的模型类（Models）。

系统利用 Django ORM 技术，将关系型数据库表映射为 Python 对象。每个模型类对应数据库中的一张表，类属性对应表字段。这种方式避免了在数据修改时直接编写复杂的 SQL 语句，确保了数据操作的安全性与可维护性。

系统重点实现了以下业务模型：

- 文旅资源模型：涵盖 POI 主表及其扩展表（景区详情、酒店详情），定义了资源的基本属性与关联关系。
- 企业信息模型：维护企业基础数据及企业与 POI 的运营关联关系。
- 动态运行数据模型：记录客流量、占用率等随时间变化的时序数据。
- 气象服务模型：包含气象位置、天气预报及气象预警信息，用于风险提示。

②后端管理功能实现（Django Admin）

为提高开发效率并便于系统维护，本项目充分利用 Django 内置的 Admin 管理后台，构建了可视化的数据管理中心。后台管理界面按功能模块对系统数据进行分组，主要包含以下功能：

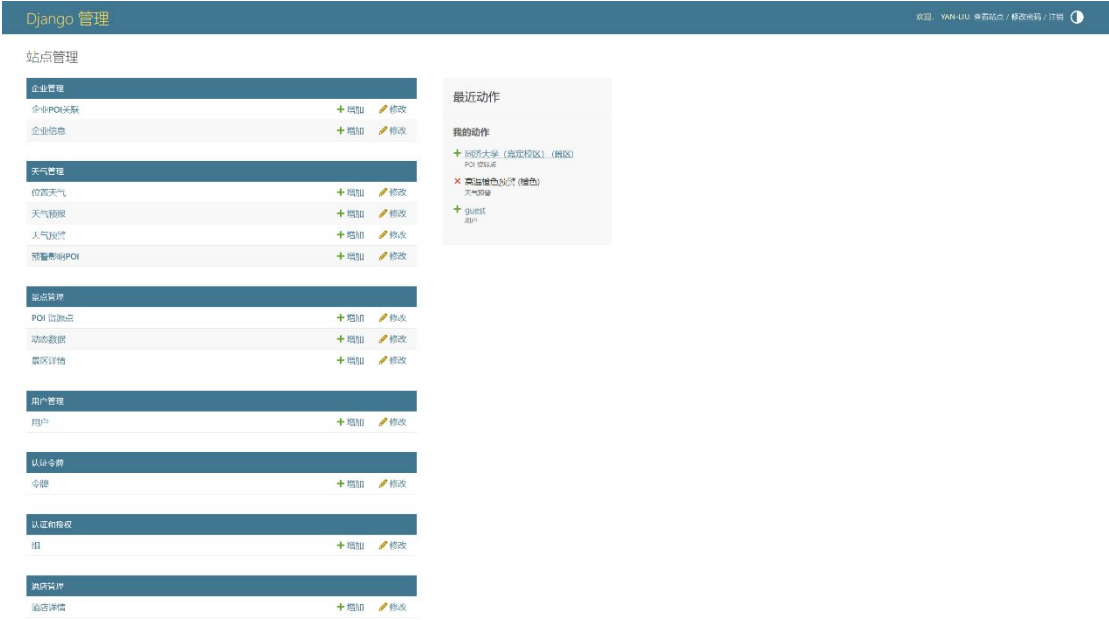


图 21 Django Admin 后台截图

基于 Django 内置的认证系统，管理员可在后台对系统用户、用户组及权限进行统一配

置。系统支持基于角色的访问控制（RBAC），管理员可设置不同用户的登录状态、超级用户权限及具体的数据表访问权限，确保系统操作的安全性。

系统将文旅资源、企业信息、动态数据及气象数据注册到 Admin 后台。管理员可通过图形化界面执行以下操作：

- 文旅资源管理：对景点和酒店的基础信息、详细属性进行增删改查。
- 企业管理：维护企业主体信息，并配置企业与特定 POI 的关联。
- 气象与动态数据：查看和修正自动同步的天气预报数据及实时的运行监测数据。

后台的所有操作均转化为对数据库的原子性操作，并受到主键、外键及完整性约束的严格限制，有效避免了非法数据的写入。同时，“最近操作记录”功能为系统审计提供了依据。

③RESTful API 接口实现

为支持前端可视化大屏及移动端的业务需求，系统基于 Django REST Framework 构建了规范的接口服务体系。接口遵循 RESTful 架构风格，支持标准的 HTTP 请求方式（GET、POST、PUT、DELETE）。系统通过序列化器（Serializer）将复杂的数据库模型实例转换为 JSON 格式数据，实现了数据的高效传输。

- POI 资源接口：提供文旅资源的列表查询、关键字搜索及详情获取功能。

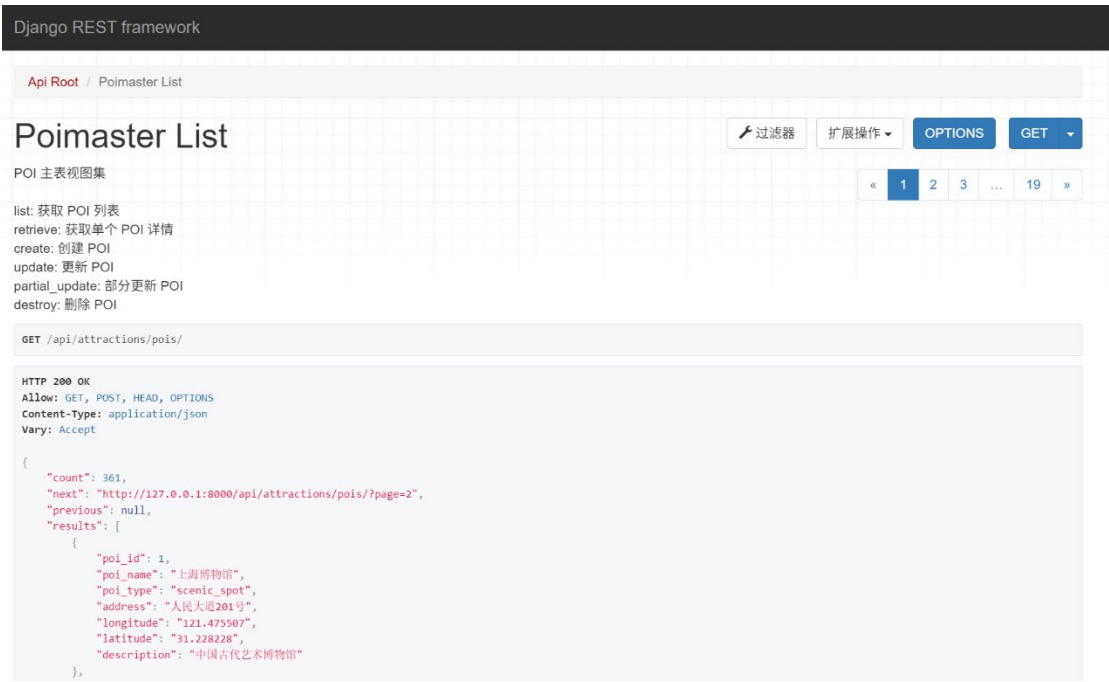


图 22 POI 资源接口（其一）RESTful 界面展示

- 动态运行数据接口：实时返回客流量、酒店入住率等监控数据，支持按时间范围查询。
- 企业与关联接口：用于获取企业的基本信息及其所管理的资源列表。
- 气象服务接口：提供特定位置的天气预报、实时天气及气象预警信息，用于前端的风险预警弹窗展示。

以上展示的 DRF 自动生成的 API 文档页面直观展示了各接口的访问路径、参数说明及返回示例，极大地便利了前后端联调与系统测试（其余接口展示均放在附录中）。

本项目通过深度整合 Django Admin 管理后台与 Django REST Framework 接口服务，成功构建了一套集数据管理、业务逻辑处理与服务分发于一体的高效后端架构。该架构以严格

的数据库设计为基石，利用 ORM 技术实现了数据模型与底层存储的无缝映射：一方面，通过自动化生成的管理后台，极大地降低了系统运维成本；另一方面，通过标准化的 RESTful API 体系，实现了前后端分离开发，为前端应用提供了灵活、统一的数据交互渠道。不仅保障了文旅资源、企业信息及气象数据的高效流转，也为系统未来的功能迭代、第三方集成以及长期稳定运行奠定了坚实的技术基础。

7.3.3 前端系统设计与实现

本系统前端采用 Vue 3（Composition API）作为核心框架，结合 Element Plus 组件库构建现代化的单页面应用（SPA）。系统设计遵循前后端分离架构，通过 RESTful API 与后端进行数据交互。

构建工具选用 Vite，利用其原生 ESM 支持实现极速的冷启动和热更新（HMR），显著提升开发效率。状态管理采用轻量级的 Pinia，取代了传统的 Vuex，以提供更好的 TypeScript 支持和更简洁的 API 设计。路由管理则依赖 Vue Router 4，实现页面间的无刷新跳转与动态路由加载。

前端工程遵循模块化开发规范，项目结构清晰，主要划分为以下核心目录：

- **src/api:** 统一存放 API 接口定义文件，将 HTTP 请求与业务逻辑解耦。
- **src/views:** 存放页面级组件（如 POI 管理页），对应路由配置中的各个视图。
- **src/components:** 存放可复用的公共组件（如布局 Layout），提高代码复用率。
- **src/store:** 基于 Pinia 的全局状态管理模块，用于存储用户信息、Token 及系统配置。
- **src/utils:** 封装工具函数，包括 Axios 请求拦截器、日期格式化等。

系统采用单文件组件(SFC)模式开发，每个组件独立包含<template>、<script>和<style>，确保了逻辑与样式的封装性。

①认证与授权模块

系统实现了基于 Token 的安全认证机制。用户在登录页（Login.vue）输入凭证，前端调用登录接口获取 Token，并将其存储在 localStorage 和 Pinia 状态树中。并且在 router/index.js 中配置全局前置守卫（beforeEach），每次路由跳转前校验 Token 有效性，未登录用户会被强制重定向至登录页，防止非法访问。



图 23 登录界面展示图

②布局与导航实现

系统采用经典的侧边栏布局结构（SidebarLayout.vue）。响应式侧边栏基于 Element Plus 的 Menu 组件，实现了多级菜单的折叠与展开，并适配不同屏幕尺寸。

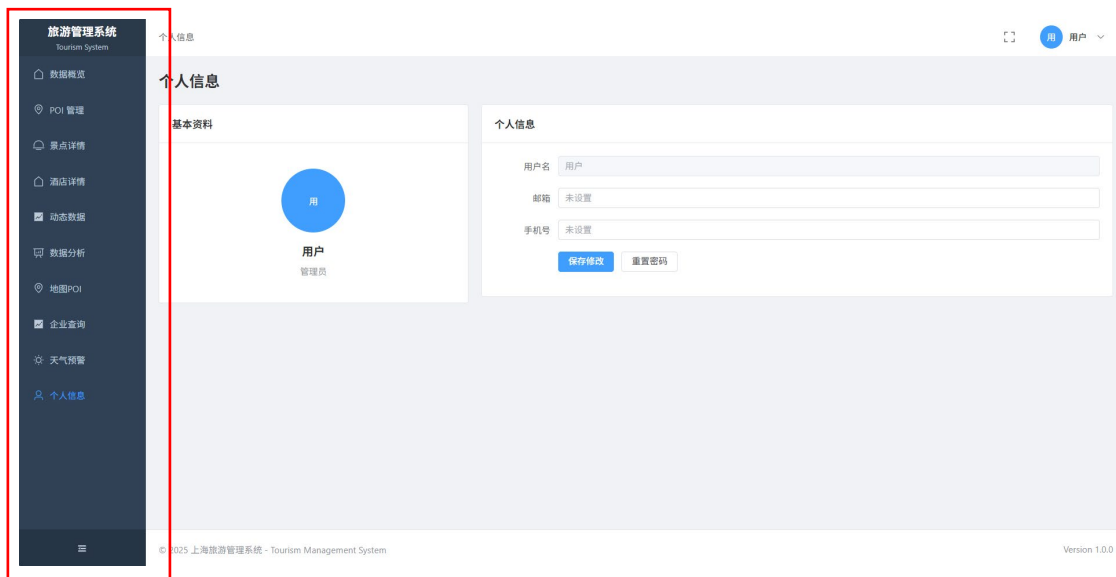


图 24 响应式侧边栏展示图

顶部导航栏集成面包屑导航（Breadcrumb）与用户信息展示，提供全屏切换与退出登录功能，增强用户交互体验。

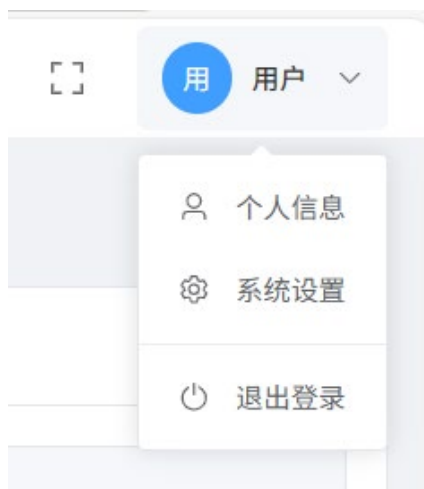


图 25 顶部导航栏展示图

③页面实现

仪表板（Dashboard.vue）作为系统首页，集中展示核心业务指标。实时显示 POI 总数、景区数量等关键数据。集成 ECharts 并利用 CSS/SVG 绘制数据饼图与排行榜，直观展示文旅资源的分布情况。页面支持数据的分页加载与状态刷新，帮助管理员快速掌握系统运行态势。

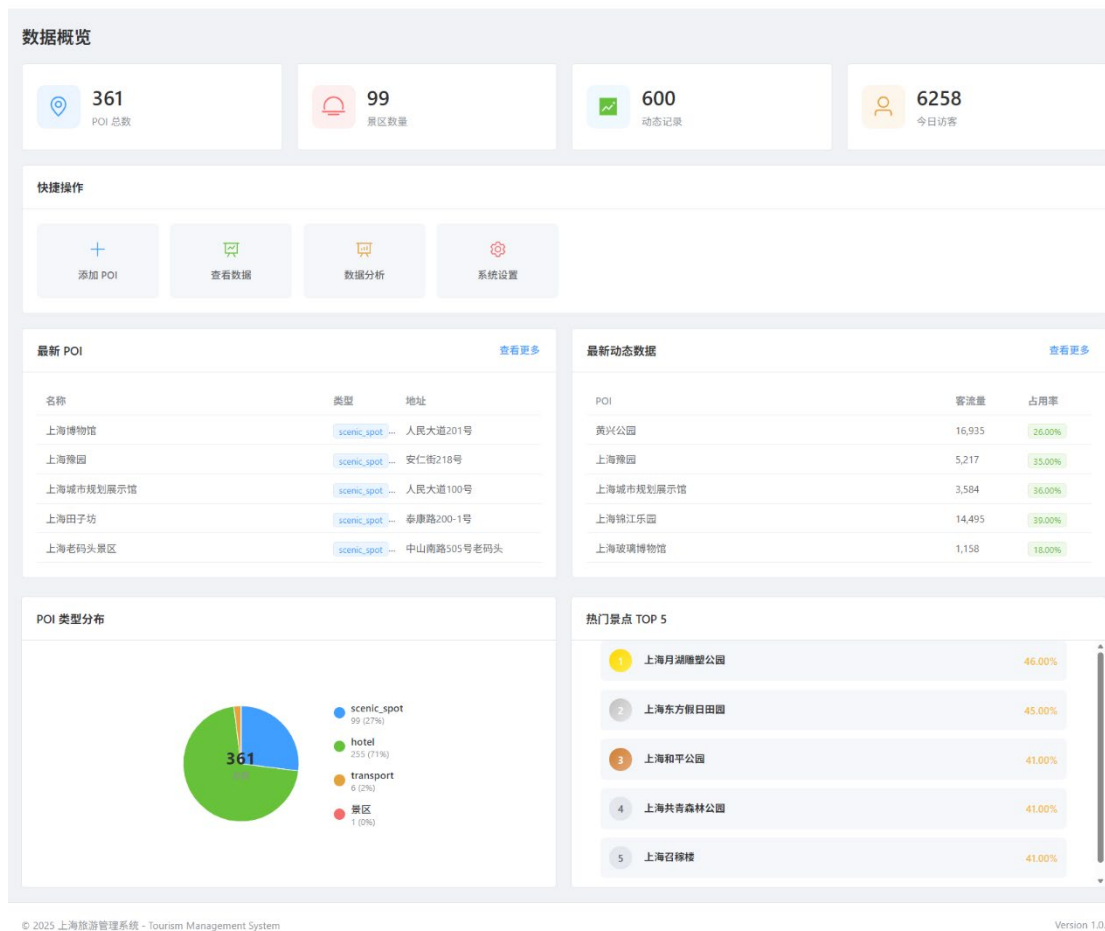


图 26 系统首页展示图

针对 POI、景点、酒店及企业信息，系统构建了标准化的管理页面（如 PoiManagement.vue）。利用 Element Plus Table 组件实现数据的列表展示、分页、排序及筛选（其余页面在附录中展示）。在新增和编辑功能中，使用 Form 组件配合自定义验证规则，确保录入数据的格式正确（如经纬度校验、必填项检查）。

添加 POI

* 名称: 请输入POI名称

* 类型: 请选择类型

地址: 请输入地址

经度: - +

纬度: - +

描述: 请输入描述

取消 确定

图 27 新增 POI 界面展示图

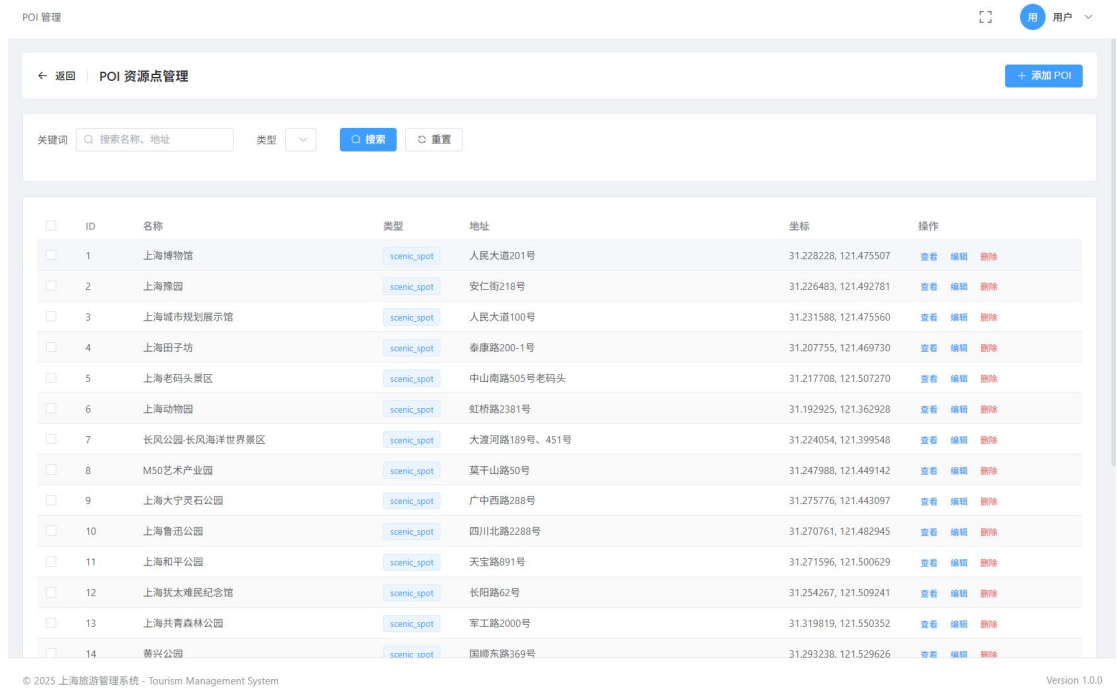


图 28 POI 资源点管理页面展示图

地理信息展示：在 MapPoiView.vue 中集成地图组件，实现 POI 点位的可视化标注与查询。

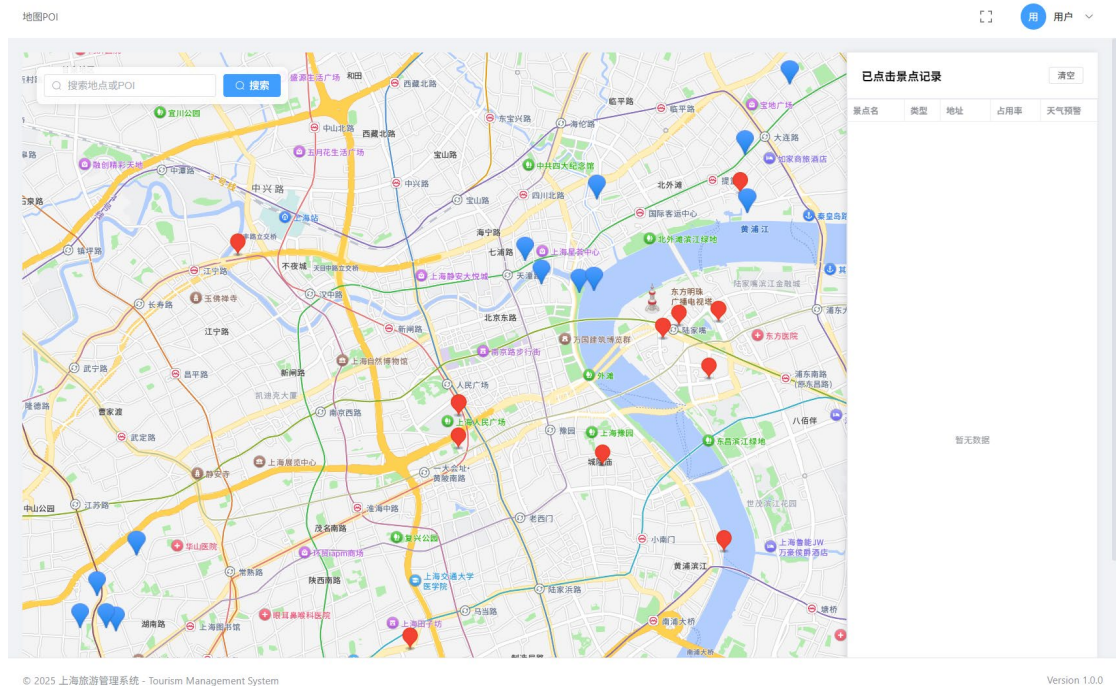


图 28 地理信息展示（地图 POI）页面展示图

关键技术实现方式如下：

➤ 网络请求封装

为统一处理 HTTP 请求，系统在 src/utlis/request.js 中对 Axios 进行了二次封装：自动在请求头（Header）中携带 Authorization: Bearer Token，免去手动添加认证信息的繁琐。并统一处理接口返回的状态码，对 401（未授权）等错误进行自动登出处理，并利用 Element Plus

在开发环境与跨域配置上，我在 `vite.config.js` 中配置开发服务器代理（Proxy），将前端的/api 请求转发至后端 Django 服务（`http://127.0.0.1:8000`）。这一配置有效解决了前后端分离开发环境下的跨域资源共享（CORS）问题，使得前后端联调更加顺畅。

总体来说，从底层的 **Axios** 请求封装到上层的路由守卫与状态管理，前端架构不仅实现了与后端 **API** 的无缝对接，还通过模块化设计保证了代码的可维护性与扩展性，为用户提供了流畅的操作体验。

系统功能测试是验证软件质量的关键环节。由于系统采用前后端分离架构，后端通过 RESTful API 向前端提供数据服务，因此接口的可用性与稳定性直接决定了系统的整体质量。在系统联调之前，本项目使用 Postman 工具对 Django REST Framework 开发的后端接口进行了独立的测试。

7.4.1 后端接口测试

我在 Postman 中建立 “Shanghai Tourism Env” 环境，配置 BaseURL 为本地开发地址 `http://127.0.0.1:8000/api`。利用 Postman 的功能，验证响应状态码是否为 200/201，以及响应体中是否包含关键字段（如 `id, name`）。

编号	请求方法	接口路径	请求参数	状态码	测试结果
API-001	POST	common/tokens/	{username: "admin", password: "..."} 	200	返回 Access Token 与 Refresh Token

编号	请求方法	接口路径	请求参数	状态码	测试结果
API-002	POST	attractions/pois/	Body: {name: "同济大学", latitude:..., ...}	200	数据创建成功，返回带 ID 的完整对象
API-003	GET	attractions/pois/	/	200	返回包含 count、next 及 results 数组的 JSON 数据
API-004	GET	orders/enterprises/	Query: page=1&size=10	200	返回固定大小包含 count、next 及 results 数组的 JSON 数据
API-005	DELETE	attractions/pois/	Header:无 Token	401	提示“身份认证信息未提供”，系统拦截成功

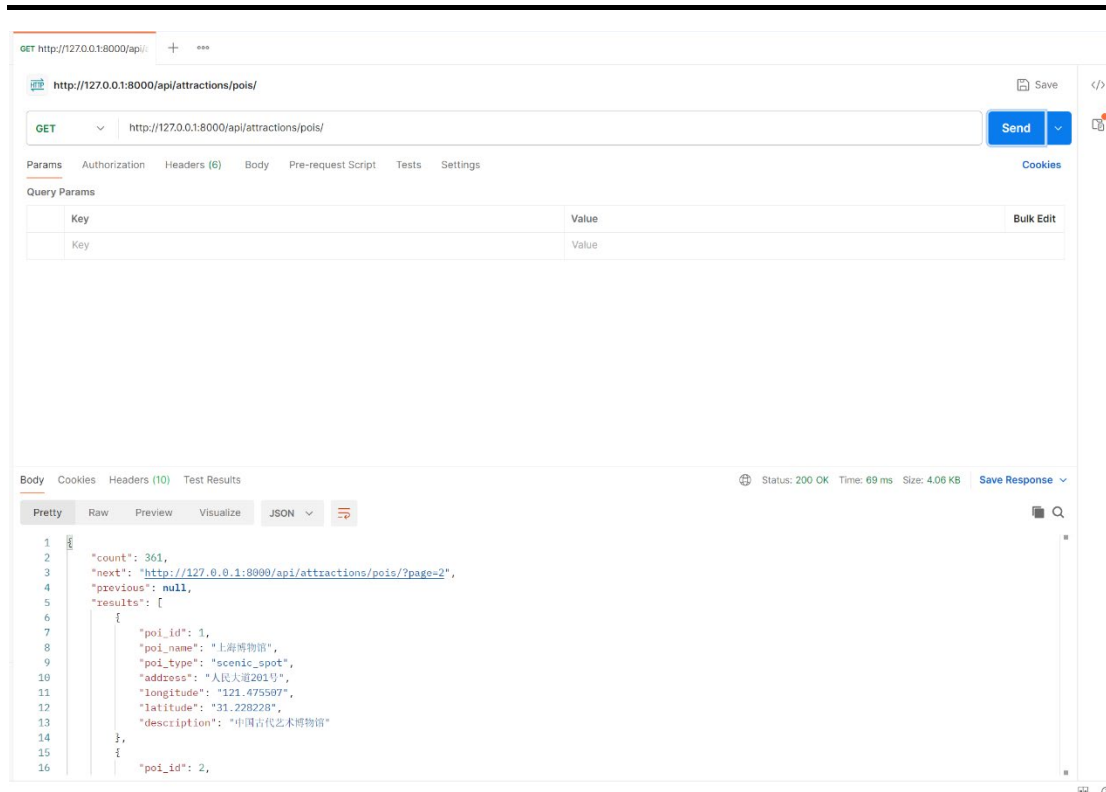


图 30 Postman 测试结果截图

7.4.2 核心模块功能测试

根据系统设计，选取了用户登录、POI 资源管理、数据可视化及企业信息管理等关键模

块进行详细测试。

表 14 用户认证与权限模块测试

编号	测试功能	操作步骤	预期结果	实际结果	测试结论
TC-001	用户登录	输入正确的用户名和密码，点击登录	提示登录成功，跳转至仪表板首页，右上角显示用户信息	200	通过
TC-002	登录失败	输入错误的用户名或密码	界面提示“用户名或密码错误”，停留在登录页	与预期一致	通过
TC-003	表单验证	用户名或密码为空时点击登录	输入框下方显示“必填项”红色提示，禁止提交请求	与预期一致	通过
TC-004	路由守卫	在未登录状态下直接访问 /dashboard URL	系统自动拦截请求，并强制重定向至登录页面	与预期一致	通过

表 15 文旅资源（POI）管理模块测试

编号	测试功能	操作步骤	预期结果	实际结果	测试结论
TC-005	POI 列表加载	进入 POI 管理页面	表格正确加载文旅资源数据，分页组件显示正常	数据加载准确，分页可用	通过
TC-006	POI 查询	在搜索框输入“外滩”，点击搜索	列表中仅显示名称包含“外滩”的资源记录	筛选结果准确	通过
TC-007	新增 POI	点击新增，填写完整信息（名称、经纬度等）并保存	提示“保存成功”，弹窗关闭，列表自动刷新并显示新数据	数据成功写入并回显	通过
TC-008	数据删除	选中一条非核心测试数据，点击删除，并在确认框点击确定	界面提示“删除成功”，该条数据从列表中消失	数据库中该记录已物理删除	通过
TC-009	经纬度校验	在新增或编辑时输入非法的经纬度格式	表单验证报错，提示坐标格式不正确	验证逻辑生效	通过

表 16 数据可视化与动态监测测试

编号	测试功能	操作步骤	预期结果	实际结果	测试结论
TC-010	仪表板展示	登录后进入首页仪表板	顶部统计卡片（POI 总数、企业数）数值准确，ECharts 图表正常渲染	显示正常，无 JS 报错	通过
TC-011	地图交互	进入地图 POI 页面，点击地图上的标记点	地图缩放至该点，并弹出包含景点详情的信息窗口	交互流畅，信息对应准确	通过
TC-012	动态数据同步	修改后端客流量数据，刷新前端页面	动态运行数据页面或图表展示的数值发生相应变化	前后端数据同步正常	通过
TC-013	天气预警显示	模拟后端发送一条气象预警数据	前端界面能正确获取并以列表或弹窗形式展示预警信息	预警信息展示及时	通过

下面给出部分有代表性的测试截图：



图 31 表单验证测试结果截图



图 32 新增 POI 测试结果截图

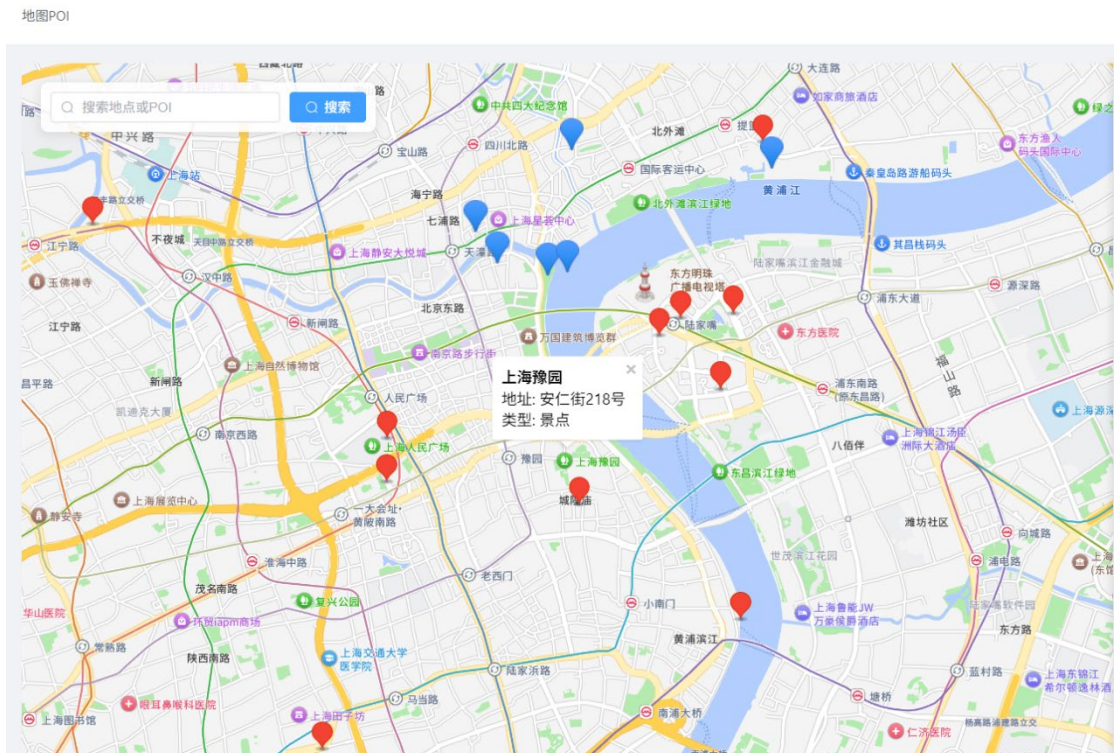


图 33 地图交互测试结果截图

经过上述多轮的功能测试，系统整体运行稳定，各项指标均达到预期目标。

八、总结

本项目针对超大城市文旅产业数字化转型中面临的多源数据分散、异构性强及关联关系复杂等痛点，以数据库系统原理为理论基石，设计并构建了上海市智慧文旅信息管理系统的底层数据架构。在概念与逻辑设计阶段，项目采用了严谨的 E-R 建模方法，构建了以“文旅资源（POI）”为核心实体，景区与酒店详情为纵向扩展，涉旅企业与气象预警为横向关联的立体化数据模型；通过引入独立的中间关联表，成功解耦了“企业—资源”运营权属及“预警—景点”受灾范围这两个复杂的多对多（M:N）业务关系，严格遵循第三范式（3NF）以消除数据冗余与传递依赖。基于 MySQL 数据库与 Django ORM 技术的深度融合，项目实现了数据完整性约束、事务处理及索引优化的落地，成功将静态的基础档案、高频更新的时序客流数据以及外部气象信息纳入统一的规范化管理体系，确立了系统“高内聚、强一致”的数据底座。

在系统开发与应用层面，本项目采用了基于 Vue.js 与 Django REST Framework 的前后端分离架构及 Docker 容器化部署方案，成功实现了一个集信息查询、动态监测与智能决策辅助于一体的综合管理平台。系统不仅完成了文旅资源全生命周期的数字化建档，更突破性地实现了数据驱动的业务闭环：通过集成 GIS 地理信息系统与 ECharts 可视化组件，将枯燥的数据库记录转化为直观的资源分布图与客流趋势大屏，支持管理部门进行“削峰填谷”的流量调度；同时，建立的气象联动机制能够在接收到预警信号时，通过后台算法自动匹配并锁定受影响的文旅场所，实现风险的精准提示。

该系统的实现不仅验证了复杂数据库模型在实际工程中的可用性，显著提升了文旅管理的科学性与应急响应速度，也为未来接入物联网（IoT）实时感知数据与开展基于历史大数据的智能预测分析预留了标准化的扩展接口，具有较高的实用价值与推广前景。

参考文献

- [1] 王珊, 杜小勇, 陈红. 数据库系统概论（第 6 版）[M]. 北京: 高等教育出版社, 2023.
- [2] 张海藩, 牟永敏. 软件工程导论（第 6 版）[M]. 北京: 清华大学出版社, 2013.
- [3] 明日科技. MySQL 从入门到精通（第 3 版）[M]. 北京: 清华大学出版社, 2023.
- [4] 黄永祥. Django 5 Web 应用开发实战 [M]. 北京: 清华大学出版社, 2024.
- [5] 储久良. Vue.js 前端框架技术与实战 [M]. 北京: 清华大学出版社, 2021.
- [6] Docker Inc. Docker Documentation [EB/OL]. <https://docs.docker.com/>, 2025.
- [7] Apache ECharts. Apache ECharts 官方文档 [EB/OL]. <https://echarts.apache.org/zh/index.html>, 2025.
- [8] 上海市大数据中心. 上海市公共数据开放平台文旅数据专区 [EB/OL]. <https://data.sh.gov.cn/>, 2025.
- [9] 高德开放平台. Web 服务 API 开发文档 [EB/OL]. <https://lbs.amap.com/api/webservice/summary>, 2025.
- [10] 中国气象局. 中国天气网气象数据服务 [EB/OL]. <http://www.weather.com.cn>, 2025.

附录

简要项目结构：

(1)后端

backend/

—— apps/	# 应用模块
—— attractions/	# 文旅资源管理
—— common/	# 公共组件
—— hotels/	# 酒店管理
—— orders/	# 订单管理
—— reviews/	# 评价与天气
—— users/	# 用户管理
—— config/	# 配置文件
—— settings/	# 环境配置
—— base.py	# 基础配置
—— dev.py	# 开发配置
—— prod.py	# 生产配置
—— urls.py	# 主路由
—— wsgi.py	
—— db.sqlite3	# SQLite 数据库文件
—— requirements.txt	# 依赖包列表
—— manage.py	# Django 管理脚本
—— .env	# 环境变量配置
—— docker-compose.yml	# Docker 配置

(2)前端

frontend/

—— public/	# 静态资源
—— src/	
—— api/	# API 接口定义
—— assets/	# 静态资源
—— components/	# 公共组件
—— Layout/	# 布局组件
—— router/	# 路由配置
—— store/	# 状态管理
—— utils/	# 工具函数
—— views/	# 页面组件
—— App.vue	# 根组件
—— main.js	# 入口文件
—— package.json	# 项目配置
—— vite.config.js	# Vite 配置

RESTful 界面展示:

Django REST framework

Api Root / Poimaster List

Poimaster List

POI 主视图图集

list: 获取 POI 列表
retrieve: 获取单个 POI 详情
create: 创建 POI
update: 更新 POI
partial_update: 部分更新 POI
destroy: 删除 POI

过滤器 扩展操作 OPTIONS GET

« 1 2 3 ... 19 »

GET /api/attractions/pois/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "count": 361,
  "next": "http://127.0.0.1:8000/api/attractions/pois/?page=2",
  "previous": null,
  "results": [
    {
      "poi_id": 1,
      "poi_name": "上海博物馆",
      "poi_type": "scenic_spot",
      "address": "人民大道201号",
      "longitude": "121.475507",
      "latitude": "31.228228",
      "description": "中国古代艺术博物馆"
    }
  ],
}
```

Django REST framework

Api Root / Scenicspotdetails List

Scenicspotdetails List

景点详情视图图集

过滤器 扩展操作 OPTIONS GET

« 1 2 3 4 5 »

GET /api/attractions/details/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "count": 99,
  "next": "http://127.0.0.1:8000/api/attractions/details/?page=2",
  "previous": null,
  "results": [
    {
      "s_id": 1,
      "poi_info": {
        "poi_id": 1,
        "poi_name": "上海博物馆",
        "poi_type": "scenic_spot",
        "address": "人民大道201号",
        "longitude": "121.475507",
        "latitude": "31.228228",
        "description": "中国古代艺术博物馆"
      },
      "level": "AAAA",
      "max_capacity": 8000,
      "scenic_spot": 1
    }
  ],
}
```

Api Root / Hoteldetails List

Hoteldetails List

酒店详情视图集

过滤器

扩展操作

OPTIONS

GET

« 1 2 3 ... 13 »

GET /api/hotels/hotels/

```
HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "count": 255,
  "next": "http://127.0.0.1:8000/api/hotels/hotels/?page=2",
  "previous": null,
  "results": [
    {
      "h_id": 1,
      "hotel_name": "上海富建酒店有限公司",
      "hotel_type": "hotel",
      "address": "七萃路1885号",
      "longitude": "121.369474",
      "latitude": "31.137714",
      "star_rating": "5星级",
      "room_count": 280
    }
  ]
}
```

Api Root / Dynamicdata List

Dynamicdata List

动态数据视图集

过滤器

扩展操作

OPTIONS

GET

« 1 2 3 ... 30 »

GET /api/attractions/dynamic/

```
HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "count": 600,
  "next": "http://127.0.0.1:8000/api/attractions/dynamic/?page=2",
  "previous": null,
  "results": [
    {
      "record_id": 168,
      "poi_name": "黄兴公园",
      "record_time": "2025-12-24 19:00:00",
      "passenger_flow": 16935,
      "occupancy_rate": "0.26",
      "poi": 14
    }
  ]
}
```


Api Root / Enterpriseinfo List

Enterpriseinfo List

企业信息视图集

list: 获取企业列表
retrieve: 获取企业详情
create: 创建企业
update: 更新企业
partial_update: 部分更新企业
destroy: 删除企业

GET /api/orders/enterprises/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
 "count": 253,
 "next": "http://127.0.0.1:8000/api/orders/enterprises/?page=2",
 "previous": null,
 "results": [
 {
 "enterprise_id": 1,
 "enterprise_name": "上海博物馆",
 "enterprise_type": "事业单位",
 "poi_count": 1
 },
],
}

Api Root / Enterprisepollink List

Enterprisepollink List

企业POI关联视图集

list: 获取关联列表
retrieve: 获取关联详情
create: 创建关联
update: 更新关联
partial_update: 部分更新关联
destroy: 删除关联

GET /api/orders/enterprise-poi-links/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
 "count": 360,
 "next": "http://127.0.0.1:8000/api/orders/enterprise-poi-links/?page=2",
 "previous": null,
 "results": [
 {
 "e_id": 1,
 "enterprise": 1,
 "enterprise_name": "上海博物馆",
 "enterprise_type": "事业单位",
 "poi": 1,
 "poi_name": "上海博物馆",
 "poi_address": "人民大道201号",
 "relationship_type": "管理"
 },
],
}

Api Root / Locationweather List

Locationweather List

过滤器

扩展操作

OPTIONS

GET

位置天气视图集

list: 获取位置天气列表
retrieve: 获取位置天气详情
create: 创建位置天气
update: 更新位置天气
partial_update: 部分更新位置天气
destroy: 删除位置天气

GET /api/reviews/locations/

```
HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "count": 16,
  "next": null,
  "previous": null,
  "results": [
    {
      "location_id": 1,
      "location_name": "黄浦区",
      "poi": null,
      "temperature": "15.80",
      "conditions": "晴",
      "update_time": "2025-12-24 10:00:00",
      "weather_summary": "晴 15.80℃"
    },
  ],
}
```

Api Root / Weatherforecast List

Weatherforecast List

过滤器

扩展操作

OPTIONS

GET

天气预报视图集

[«](#) [1](#) [2](#) [3](#) [...](#) [6](#) [»](#)

list: 获取天气预报列表
retrieve: 获取天气预报详情
create: 创建天气预报
update: 更新天气预报
partial_update: 部分更新天气预报
destroy: 删除天气预报

GET /api/reviews/forecasts/

```
HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "count": 112,
  "next": "http://127.0.0.1:8000/api/reviews/forecasts/?page=2",
  "previous": null,
  "results": [
    {
      "forecast_id": 1,
      "location": 1,
      "location_name": "黄浦区",
      "forecast_date": "2025-12-25",
      "temp_max": "19.10",
      "temp_min": "12.40",
      "condition_day": "多云"
    },
  ],
}
```

Django REST framework

Api Root / Weatheralert List

Weatheralert List

天气预警视图集

list: 获取天气预警列表
retrieve: 获取天气预警详情
create: 创建天气预警
update: 更新天气预警
partial_update: 部分更新天气预警
destroy: 删除天气预警

GET /api/reviews/alerts/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
 "count": 4,
 "next": null,
 "previous": null,
 "results": [
 {
 "alert_id": 5,
 "title": "寒潮蓝色预警",
 "level": "蓝色",
 "status": "预警中",
 "publish_time": "2025-12-24 12:00:00",
 "affected_poi_count": 14,
 "is_active": false
 },
],
}

Django REST framework

Api Root / Alertaffectedpoi List

Alertaffectedpoi List

预警影响POI视图集

list: 获取预警影响POI列表
retrieve: 获取预警影响POI详情
create: 创建预警影响POI关联
update: 更新预警影响POI关联
partial_update: 部分更新预警影响POI关联
destroy: 删除预警影响POI关联

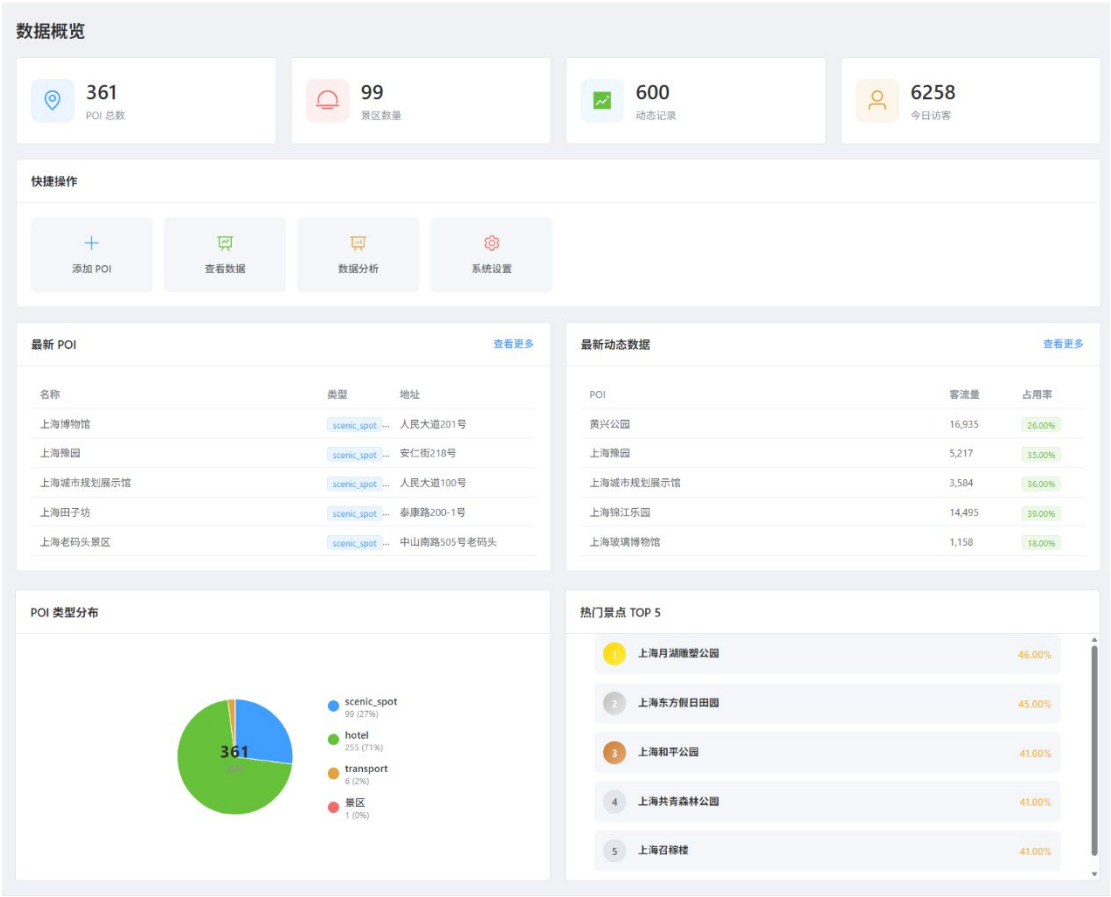
GET /api/reviews/alert-affected-pois/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
 "count": 41,
 "next": "http://127.0.0.1:8000/api/reviews/alert-affected-pois/?page=2",
 "previous": null,
 "results": [
 {
 "a_id": 1,
 "alert": 1,
 "alert_title": "暴雨黄色预警",
 "alert_level": "黄色",
 "alert_status": "生效中",
 "poi": 126,
 "poi_name": "上海紫泰酒店管理有限公司",
 "poi_address": "虹桥路2270号"
 },
],
}

« 1 2 3 »

前端页面面展示：



POI 管理

用户

POI 资源点管理

+ 添加 POI

关键词

搜索名称、地址

类型

搜索

重置

☐	ID	名称	类型	地址	坐标	操作
☐	1	上海博物馆	scenic_spot	人民大道201号	31.228228, 121.475507	查看 编辑 删除
☐	2	上海豫园	scenic_spot	安仁街218号	31.226483, 121.492781	查看 编辑 删除
☐	3	上海城市规划展示馆	scenic_spot	人民大道100号	31.231588, 121.475560	查看 编辑 删除
☐	4	上海田子坊	scenic_spot	泰康路200-1号	31.207755, 121.469730	查看 编辑 删除
☐	5	上海老码头景区	scenic_spot	中山南路505号老码头	31.217708, 121.507270	查看 编辑 删除
☐	6	上海动物园	scenic_spot	虹桥路2381号	31.192925, 121.362928	查看 编辑 删除
☐	7	长风公园-长风海洋世界景区	scenic_spot	大渡河路189号、451号	31.224054, 121.399548	查看 编辑 删除
☐	8	M50艺术产业园	scenic_spot	莫干山路50号	31.247988, 121.449142	查看 编辑 删除
☐	9	上海大宁灵石公园	scenic_spot	广中西路288号	31.275776, 121.443097	查看 编辑 删除
☐	10	上海鲁迅公园	scenic_spot	四川北路2288号	31.270761, 121.482945	查看 编辑 删除
☐	11	上海和平公园	scenic_spot	天宝路891号	31.271596, 121.500629	查看 编辑 删除
☐	12	上海犹太难民纪念馆	scenic_spot	长阳路62号	31.254267, 121.509241	查看 编辑 删除
☐	13	上海共青森林公园	scenic_spot	军工路2000号	31.319819, 121.550352	查看 编辑 删除
☐	14	复兴公园	scenic_spot	国顺东路369号	31.293238, 121.529626	查看 编辑 删除

© 2025 上海旅游管理系统 - Tourism Management System

Version 1.0.0

景点详情

用户

景点详情管理

ID	景区名称	级别	最大承载量	操作
1	上海博物馆	AAAA	8,000	查看 编辑
2	上海豫园	AAAA	15,000	查看 编辑
3	上海城市规划展示馆	AAAA	10,000	查看 编辑
4	上海田子坊	AAA	8,000	查看 编辑
5	上海老码头景区	AAA	23,744	查看 编辑
6	上海动物园	AAAA	44,000	查看 编辑
7	长风公园-长风海洋世界景区	AAAA	50,000	查看 编辑
8	M50艺术产业园	AAA	14,350	查看 编辑
9	上海大宁灵石公园	AAA	105,247	查看 编辑
10	上海鲁迅公园	AAA	102,400	查看 编辑
11	上海和平公园	AAA	89,731	查看 编辑
12	上海犹太难民纪念馆	AAA	1,600	查看 编辑
13	上海共青森林公园	AAAA	63,000	查看 编辑
14	复兴公园	AAA	65,000	查看 编辑
15	上海锦江乐园	AAAA	37,500	查看 编辑
16	上海召稼楼	AAA	42,000	查看 编辑
17	上海老外街区	AAA	11,692	查看 编辑

© 2025 上海旅游管理系统 - Tourism Management System

Version 1.0.0

地图POI

搜索地点或POI

搜索

上海东方明珠广播电视塔

地址: 世纪大道1号

类型: 景点

已点击景点记录

清空

景点名	类型	地址	占用率	天气预警
上海东方明珠广播电视塔	景点	世纪大道1号	27.00%	无预警

© 2025 上海旅游管理系统 - Tourism Management System

Version 1.0.0

企业查询

企业查询管理

刷新

企业名称

输入企业名称

企业类型

搜索

重置

新增企业

企业列表

ID	企业名称	企业类型	操作
1	上海博物馆	事业单位	查看 编辑 POI关联 删除
2	上海豫园旅游商城股份有限公司	上市公司	查看 编辑 POI关联 删除
3	上海市规划和自然资源局	政府机构	查看 编辑 POI关联 删除
4	田子坊文化发展有限公司	民营企业	查看 编辑 POI关联 删除
5	上海老码头投资管理有限公司	民营企业	查看 编辑 POI关联 删除
6	上海动物园	事业单位	查看 编辑 POI关联 删除
7	上海长风海洋世界	民营企业	查看 编辑 POI关联 删除
8	M50创意园区管理公司	民营企业	查看 编辑 POI关联 删除
9	上海静安区绿化市容局	政府机构	查看 编辑 POI关联 删除
10	上海虹口区绿化市容局	政府机构	查看 编辑 POI关联 删除
11	上海虹口区文旅局	政府机构	查看 编辑 POI关联 删除
12	上海共青森林公园	事业单位	查看 编辑 POI关联 删除
13	上海杨浦区绿化市容局	政府机构	查看 编辑 POI关联 删除
14	锦江国际集团	国有企业	查看 编辑 POI关联 删除

© 2025 上海旅游管理系统 - Tourism Management System

Version 1.0.0

天气预报与预警管理

城市

选择城市

预警级别

预警状态

搜索

重置

新增预警

黄浦区

15.80°C

()

更新时间: 2025/12/24 10:00:00

详情

虹口区

15.30°C

()

更新时间: 2025/12/24 10:00:00

详情

嘉定区

14.70°C

()

更新时间: 2025/12/24 10:00:00

详情

青浦区

14.40°C

()

更新时间: 2025/12/24 10:00:00

详情

长宁区

15.20°C

()

更新时间: 2025/12/24 10:00:00

详情

杨浦区

15.10°C

()

更新时间: 2025/12/24 10:00:00

详情

浦东新区

15.50°C

()

更新时间: 2025/12/24 10:00:00

详情

奉贤区

14.20°C

()

更新时间: 2025/12/24 10:00:00

详情

普陀区

14.90°C

()

更新时间: 2025/12/24 10:00:00

详情

闵行区

14.80°C

()

更新时间: 2025/12/24 10:00:00

详情

金山区

14.30°C

()

更新时间: 2025/12/24 10:00:00

详情

崇明区

13.80°C

()

更新时间: 2025/12/24 10:00:00

详情

静安区

15.60°C

()

更新时间: 2025/12/24 10:00:00

详情

宝山区

14.50°C

()

更新时间: 2025/12/24 10:00:00

详情

松江区

14.60°C

()

更新时间: 2025/12/24 10:00:00

详情

徐汇区

15.00°C

()

更新时间: 2025/12/24 10:00:00

详情

天气预警列表

刷新

ID	预警标题	预警级别	状态	发布时间	操作
5	寒潮蓝色预警	蓝色	预警中	2025/12/24 12:00:00	查看 受影响POI 删除
3	雷电黄色预警	黄色	已解除	2025/12/24 10:00:00	查看 受影响POI 激活 删除
2	大风蓝色预警	蓝色	生效中	2025/12/24 09:00:00	查看 编辑 受影响POI 删除
1	暴雨黄色预警	黄色	生效中	2025/12/24 08:00:00	查看 编辑 受影响POI 删除

共 4 条

20条/页

< 1 >

前往

1

页

SQL 语句设计:

```
CREATE DATABASE IF NOT EXISTS tourism_db
    DEFAULT CHARACTER SET utf8mb4
    DEFAULT COLLATE utf8mb4_general_ci;
```

```
USE tourism_db;
```

-- 1. PoiMaster (文旅资源主表)

```
CREATE TABLE PoiMaster (
    poi_id INT PRIMARY KEY COMMENT '全局唯一资源点ID',
    poi_name VARCHAR(255) NOT NULL,
    poi_type VARCHAR(100) NOT NULL,
    address VARCHAR(500),
    longitude DECIMAL(10,6),
    latitude DECIMAL(10,6),
    description TEXT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 2. ScenicSpotDetails (景区扩展)

```
CREATE TABLE ScenicSpotDetails (
    s_id INT PRIMARY KEY AUTO_INCREMENT,
    scenic_spot_id INT NOT NULL COMMENT '景区ID=poi_id',
    level VARCHAR(10),
    max_capacity INT,
    UNIQUE KEY uq_scenic_spot (scenic_spot_id),
    FOREIGN KEY (scenic_spot_id) REFERENCES PoiMaster(poi_id)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 3. HotelDetails (酒店扩展)

```
CREATE TABLE HotelDetails (
    h_id INT PRIMARY KEY AUTO_INCREMENT,
    hotel_id INT NOT NULL COMMENT '酒店ID=poi_id',
    scenic_spot_detail_id INT NULL COMMENT '所属景区扩展ID',
    star_rating VARCHAR(50),
    room_count INT,
    UNIQUE KEY uq_hotel (hotel_id),
    FOREIGN KEY (hotel_id) REFERENCES PoiMaster(poi_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (scenic_spot_detail_id) REFERENCES ScenicSpotDetails(s_id)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 4. EnterpriseInfo

```
CREATE TABLE EnterpriseInfo (
    enterprise_id INT PRIMARY KEY,
    enterprise_name VARCHAR(255),
    enterprise_type VARCHAR(255)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```

-- 5. EnterprisePoiLink
CREATE TABLE EnterprisePoiLink (
    e_id INT PRIMARY KEY AUTO_INCREMENT,
    enterprise_id INT NOT NULL,
    poi_id INT NOT NULL,
    relationship_type VARCHAR(255),
    UNIQUE KEY uq_enterprise_poi (enterprise_id, poi_id),
    FOREIGN KEY (enterprise_id) REFERENCES EnterpriseInfo(enterprise_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (poi_id) REFERENCES PoiMaster(poi_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 6. DynamicData (动态数据)
CREATE TABLE DynamicData (
    record_id INT PRIMARY KEY,
    poi_id INT NOT NULL,
    record_time DATETIME NOT NULL,
    passenger_flow INT,
    occupancy_rate DECIMAL(5,2),
    FOREIGN KEY (poi_id) REFERENCES PoiMaster(poi_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 7. LocationWeather
CREATE TABLE LocationWeather (
    location_id INT PRIMARY KEY,
    location_name VARCHAR(255),
    longitude DECIMAL(10,6),
    latitude DECIMAL(10,6),
    poi_id INT NULL,
    temperature DECIMAL(5,2),
    conditions VARCHAR(100),
    update_time DATETIME,
    FOREIGN KEY (poi_id) REFERENCES PoiMaster(poi_id)
        ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 8. WeatherForecast
CREATE TABLE WeatherForecast (
    forecast_id INT PRIMARY KEY,
    location_id INT NOT NULL,
    forecast_date DATE,
    temp_max DECIMAL(5,2),
    temp_min DECIMAL(5,2),
    condition_day VARCHAR(100),
    FOREIGN KEY (location_id) REFERENCES LocationWeather(location_id)
        ON DELETE CASCADE ON UPDATE CASCADE
)

```

```

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 9. WeatherAlert
CREATE TABLE WeatherAlert (
    alert_id INT PRIMARY KEY,
    title VARCHAR(255),
    level VARCHAR(50),
    status VARCHAR(50),
    publish_time DATETIME
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 10. AlertAffectedPoi
CREATE TABLE AlertAffectedPoi (
    a_id INT PRIMARY KEY AUTO_INCREMENT,
    alert_id INT NOT NULL,
    poi_id INT NOT NULL,
    UNIQUE KEY uq_alert_poi (alert_id, poi_id),
    FOREIGN KEY (alert_id) REFERENCES WeatherAlert(alert_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (poi_id) REFERENCES PoiMaster(poi_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```